OCOM510M Data Science: Assessment 2

Student ID: 202027438

==========

1. Aims, objectives and plan

==========

Aim

This project uses historical insurance fraud data to create a model that helps an insurance company identify potential fraudulent claims, aiming to reduce financial losses and prevent legitimate claims from being wrongly flagged.

Objectives

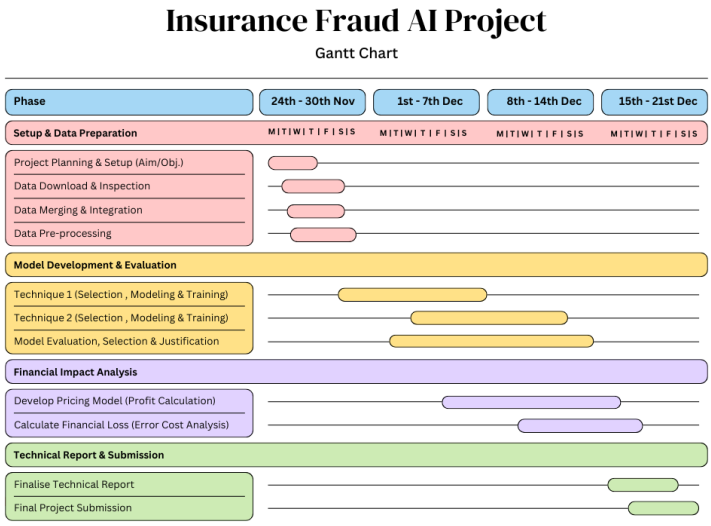
The following objectives will steer us towards achieving the goal of this project:

1. **Data acquisition and Preparation:**
    - Merge, inspect, and integrate the data files into a single dataset
    - Perform necessary pre-processing to deal with missing values
2. **Model development and Justification:**
    - Select two machine learning techniques to classify or predict customer fraud
    - Use RapidMiner to prototype the workflow, validating the approach before implementing in Python
3. **Performance evaluation and Reporting:**
    - Train and evaluate models using appropriate data splits, such as cross-validation
    - Assess performance with confusion matrices, Balanced Accuracy, F1-score, Recall and ROC curves
4. **Financial Impact Analysis (Cost-Benefit) Modelling:**
    - Create a pricing model based on average claim amount, gross profit goal, and claim frequency to estimate policy price and customer base

- Use the model's error metrics, including false positives and false negatives, to calculate the company's total financial loss from prediction errors

Plan: Simple Gantt chart

This demonstrates how the project was conducted using a simple Gantt chart.



2. Understanding the case study

Case study analysis

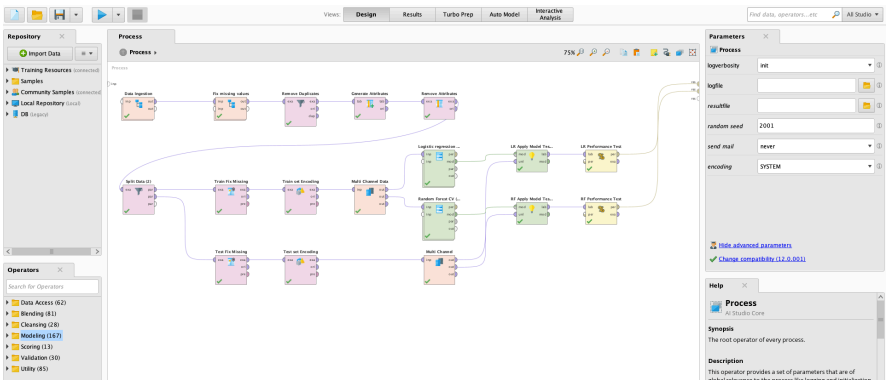
Key points identified in the case and how they are addressed to meet the client's needs.

#	Key Point from the Case Study	Intended Approach to Address the Client's Need
1	**Client's Objective**: Predict the likelihood of a customer or claim being fraudulent.	Apply two suitable classification methods (such as Logistic Regression, Decision Trees, or Random Forest) to the dataset. The target variable will be the fraud indicator.
2	**Incorrectly classifying genuine Claims**: False positives in identifying issues cause customer loss and high costs.	Prioritising precision and specificity to reduce false positives and referrals of legitimate claims.
3	**Incorrectly Approving Fraudulent Claims**: Fraudulent claims result in	Prioritise high recall to effectively identify genuine fraud cases.

#	Key Point from the Case Study	Intended Approach to Address the Client's Need
	financial loss.	
4	**Target Performance Metrics**: The client requires a model with 95% balanced accuracy.	Balanced accuracy will be the primary evaluation metric, as it addresses class imbalance and equally assesses fraud and non-fraud performance.
5	**Financial Projection Model**: Quantify the model error loss using a pricing model based on average claims, profit targets, and claim rates.	Develop and implement a pricing model that translates prediction errors into financial costs for business insights.

Rapid Miner Prototyping

A screenshot of the RapidMiner prototyping for this project is shown below.



==========

3. Pre-processing applied

==========

Import libraries & CSV Data

Import libraries

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, Gr
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report,
                             balanced_accuracy_score, f1_score,
                             precision_score, recall_score, roc_auc_score
```

```
                                roc_curve, precision_recall_curve, average_p
from sklearn.feature_selection import RFE, SelectKBest, f_classif, Varian
from sklearn.impute import SimpleImputer
from sklearn.decomposition import PCA
import warnings
warnings.filterwarnings('ignore')
```

Set random seed for reproducibility

In [2]:
```
np.random.seed(42)
```

Loading data from csv files

In [3]:
```
demographics_df = pd.read_csv('./train_data/Train_Demographics.csv')
policy_df = pd.read_csv('./train_data/Train_Policy.csv')
claim_df = pd.read_csv('./train_data/Train_Claim.csv')
vehicle_df = pd.read_csv('./train_data/Train_Vehicle.csv')
fraud_df = pd.read_csv('./train_data/Traindata_with_Target.csv')

# Convert the ReportedFraud column from categorical (Y/N) to binary (1/0)
fraud_df['ReportedFraud'] = fraud_df['ReportedFraud'].map({'Y': 1, 'N': 0
```

Shape of Datasets

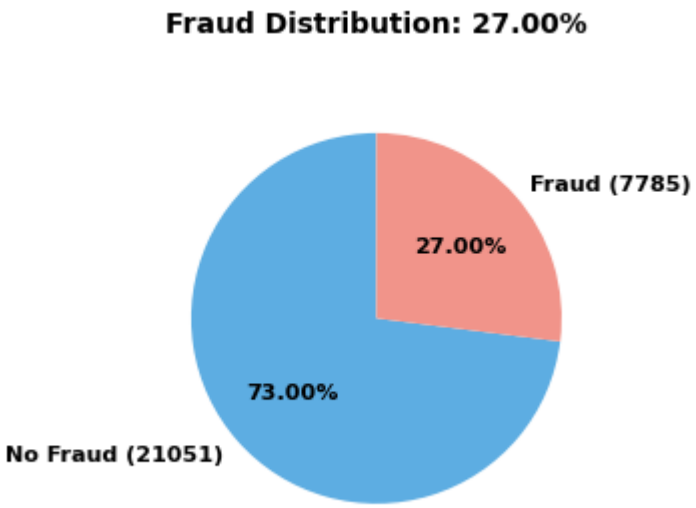
In [4]:
```
print("- Demographics:", demographics_df.shape)
print("- Policy:", policy_df.shape)
print("- Claims:", claim_df.shape)
print("- Vehicle:", vehicle_df.shape)
print("- Fraud Labels:", fraud_df.shape)
```

```
- Demographics: (28836, 10)
- Policy: (28836, 10)
- Claims: (28836, 19)
- Vehicle: (115344, 3)
- Fraud Labels: (28836, 2)
```

Fraud Distribution

In [5]:
```
fraud_counts = fraud_df['ReportedFraud'].value_counts()
labels = [f'No Fraud ({fraud_counts[0]})', f'Fraud ({fraud_counts[1]})']
colors = ['#5dade2', '#f1948a']

plt.figure(figsize=(5, 3))
plt.pie(fraud_counts, labels=labels, autopct='%1.2f%%', colors=colors,
        startangle=90, textprops={'fontsize': 8, 'weight': 'bold'})
plt.title(f"Fraud Distribution: {fraud_df['ReportedFraud'].mean()*100:.2f
plt.tight_layout(); plt.show()
```



Merging, pivoting and melting

Merge all datasets on CustomerID

In [6]:
```
dfs = [demographics_df, policy_df, claim_df, vehicle_df, fraud_df]
df = dfs[0]
for d in dfs[1:]:
    df = df.merge(d, on='CustomerID', how='inner')

print(f"\nMerged dataset shape: {df.shape}")
print(f"Total features: {df.shape[1] - 1} (excluding target)")
```

```
Merged dataset shape: (115344, 40)
Total features: 39 (excluding target)
```

Preparing the labels appropriately

Target variable: 'ReportedFraud'

In [7]:
```
print("\nTarget type: Binary classification (0 = Legitimate, 1 = Fraud)",
      "\nClass distribution:",
      df['ReportedFraud'].value_counts())
```

```
Target type: Binary classification (0 = Legitimate, 1 = Fraud)
Class distribution: ReportedFraud
0    84204
1    31140
Name: count, dtype: int64
```

Dealing with missing values (imputation, filtering) without leaking

Handle missing value placeholder

In [8]:
```
missing_map = {'InsuredGender': 'NA', 'TypeOfCollission': '?', 'PropertyD
               'PoliceReport': '?', 'Witnesses': 'MISSINGVALUE',
               'AmountOfTotalClaim': 'MISSEDDATA', 'VehicleAttributeDetai
               'ContractType': 'NA', 'PolicyAnnualPremium': -1, 'Incident
```

```
df.replace(missing_map, np.nan, inplace=True)
```

Convert numeric string columns

In [9]:
```
df['AmountOfTotalClaim'] = pd.to_numeric(df['AmountOfTotalClaim'], errors
df['Witnesses'] = pd.to_numeric(df['Witnesses'], errors='coerce')
```

In [10]:
```
df
```

Out[10]:

	CustomerID	InsuredAge	InsuredZipCode	InsuredGender	InsuredEducatio
0	Cust10000	35	454776	MALE	
1	Cust10000	35	454776	MALE	
2	Cust10000	35	454776	MALE	
3	Cust10000	35	454776	MALE	
4	Cust10001	36	454776	MALE	
...	...	...	...	...	
115339	Cust9997	53	472634	MALE	As
115340	Cust9999	36	450730	FEMALE	
115341	Cust9999	36	450730	FEMALE	
115342	Cust9999	36	450730	FEMALE	
115343	Cust9999	36	450730	FEMALE	

115344 rows × 40 columns

Convert dates

In [11]:
```
date_columns = ['DateOfPolicyCoverage', 'DateOfIncident']
for col in date_columns:
    df[col] = pd.to_datetime(df[col])
```

Missing values identified

In [12]:
```
missing_summary = df.isnull().sum()
missing_pct = (missing_summary / len(df) * 100).round(2)
missing_df = pd.DataFrame({'Count': missing_summary, 'Percentage': missin
print(missing_df[missing_df['Count'] > 0].sort_values('Count', ascending=
```

```
                          Count  Percentage
PropertyDamage            41836       36.27
PoliceReport              39220       34.00
TypeOfCollission          20648       17.90
AuthoritiesContacted      10768        9.34
PolicyAnnualPremium         564        0.49
AmountOfTotalClaim          200        0.17
Witnesses                   184        0.16
IncidentTime                124        0.11
InsuredGender               120        0.10
VehicleAttributeDetails      50        0.04
Country                       8        0.01
```

Dealing with duplicate values

Count and remove duplicates

In [13]:
```
duplicates = df['CustomerID'].duplicated().sum()
print('\nDuplicate CustomerIDs:', duplicates)

df = df.drop_duplicates('CustomerID') if duplicates else df
print('Duplicates removed' if duplicates else 'No duplicates', end='\n\n'
print('Shape:', df.shape)
```

```
Duplicate CustomerIDs: 86508
Duplicates removed

Shape: (28836, 40)
```

Feature engineering

Temporal features

In [14]:
```
df['PolicyDuration'] = (df['DateOfIncident'] - df['DateOfPolicyCoverage']
df['ClaimDelay'] = (pd.Timestamp.now() - df['DateOfIncident']).dt.days
```

Financial ratios

In [15]:
```
df['ClaimToPremiumRatio'] = df['AmountOfTotalClaim'] / (df['PolicyAnnualP
df['InjuryToTotalRatio'] = df['AmountOfInjuryClaim'] / (df['AmountOfTotal
```

Interaction features

In [16]:
```
severity_map = {'Minor': 1, 'Major': 2, 'Total Loss': 3}
df['SeverityNumeric'] = df['SeverityOfIncident'].map(severity_map)
df['SeverityXClaim'] = df['SeverityNumeric'] * df['AmountOfVehicleDamage'
```

New features created

- PolicyDuration
- CustomerTenure
- ClaimDelay
- ClaimToPremiumRatio
- InjuryToTotalRatio
- SeverityXClaim

Drop unnecessary columns

In [17]:
```
cols_to_drop = ['CustomerID', 'InsurancePolicyNumber', 'DateOfPolicyCover
                'DOE', 'DateOfIncident', 'IncidentAddress', 'InsuredZipCo
df_processed = df.drop(columns=[col for col in cols_to_drop if col in df.

print(f" Dataset shape after feature engineering: {df_processed.shape}")
```

```
Dataset shape after feature engineering: (28836, 40)
```

Train-test split

Preparing features and labels

In [18]:
```
X = df_processed.drop('ReportedFraud', axis=1)
y = df_processed['ReportedFraud']
```

Leakage check

In [19]:
```
# Leakage check
suspicious_keywords = ['fraud', 'reportedfraud', 'target', 'label', 'outc
suspicious_by_name = [col for col in X.columns if any(k in col.lower() fo

print("\nPotential leakage columns (name-based):")
print(suspicious_by_name or "  None found")

# Correlation-based leakage scan
X_num = X.select_dtypes(include=['int64', 'float64'])
if not X_num.empty:
    high_corr = X_num.join(y).corr(numeric_only=True)['ReportedFraud'].dr
    high_corr = high_corr[high_corr >= 0.98]
    print("\nPotential leakage columns (|corr| >= 0.98 with target):")
    print(high_corr if len(high_corr) > 0 else "  None found")
else:
    print("\nNo numeric columns for correlation scan.")
```

```
Potential leakage columns (name-based):
  None found

Potential leakage columns (|corr| >= 0.98 with target):
  None found
```

Identify numerical and categorical columns

In [20]:
```
numerical_features = X.select_dtypes(include=['int64', 'float64']).column
categorical_features = X.select_dtypes(include=['object']).columns.tolist

print(f"\n Numerical features: {len(numerical_features)}")
print(f" Categorical features: {len(categorical_features)}")
```

```
Numerical features: 21
Categorical features: 18
```

Stratified train-test split

In [21]:
```
X_train, X_test, y_train, y_test= train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
```

```
)
print(f"\nTrain: {X_train.shape}, {y_train.mean()*100:.2f}%")
print(f"Test: {X_test.shape}, {y_test.mean()*100:.2f}%")
```

```
Train: (20185, 39), 27.00%
Test: (8651, 39), 27.00%
```

Missing value indicator features

In [22]:
```
X_train = X_train.copy()
X_test = X_test.copy()

missing_indicator_cols = []
for col in X_train.columns:
    if X_train[col].isna().any():
        ind_col = f"{col}__missing"
        X_train[ind_col] = X_train[col].isna().astype(int)
        X_test[ind_col] = X_test[col].isna().astype(int)
        missing_indicator_cols.append(ind_col)

# Treat indicators as numerical features downstream
numerical_features = numerical_features + missing_indicator_cols

print(f"\n Missing-indicator columns created: {len(missing_indicator_cols
if len(missing_indicator_cols) > 0:
    print(f" Sample indicators: {missing_indicator_cols[:10]}")
```

```
 Missing-indicator columns created: 14
 Sample indicators: ['InsuredGender__missing', 'PolicyAnnualPremium__missi
ng', 'TypeOfCollission__missing', 'AuthoritiesContacted__missing', 'Incide
ntTime__missing', 'PropertyDamage__missing', 'Witnesses__missing', 'Police
Report__missing', 'AmountOfTotalClaim__missing', 'VehicleAttributeDetails_
_missing']
```

Missing value imputation (without leaking)

Numerical imputation using median

In [23]:
```
num_imputer = SimpleImputer(strategy='median')
X_train_num = X_train[numerical_features].copy()
X_test_num = X_test[numerical_features].copy()

X_train_num_imputed = pd.DataFrame(
    num_imputer.fit_transform(X_train_num),
    columns=numerical_features,
    index=X_train_num.index
)

X_test_num_imputed = pd.DataFrame(
    num_imputer.transform(X_test_num),
    columns=numerical_features,
    index=X_test_num.index
)
```

Categorical imputation using most frequent

In [24]:
```
cat_imputer = SimpleImputer(strategy='most_frequent')
X_train_cat = X_train[categorical_features].copy()
X_test_cat = X_test[categorical_features].copy()
```

```
X_train_cat_imputed = pd.DataFrame(
    cat_imputer.fit_transform(X_train_cat),
    columns=categorical_features,
    index=X_train_cat.index
)
X_test_cat_imputed = pd.DataFrame(
    cat_imputer.transform(X_test_cat),
    columns=categorical_features,
    index=X_test_cat.index
)
```

Identifying and dealing with outliers

Function to capture to prevent model distortion

In [25]:
```
def cap_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df[column] = df[column].clip(lower=lower_bound, upper=upper_bound)
    return df
```

Features with outlier treatment

In [26]:
```
claim_features = ['AmountOfVehicleDamage', 'AmountOfInjuryClaim',
                  'AmountOfPropertyClaim', 'AmountOfTotalClaim']

outlier_count = 0
for col in claim_features:
    if col in numerical_features:
        before_min = X_train_num_imputed[col].min()
        before_max = X_train_num_imputed[col].max()

        X_train_num_imputed = cap_outliers(X_train_num_imputed, col)
        X_test_num_imputed = cap_outliers(X_test_num_imputed, col)

        after_min = X_train_num_imputed[col].min()
        after_max = X_train_num_imputed[col].max()

        if before_min != after_min or before_max != after_max:
            outlier_count += 1
            print(f" {col}: Outliers capped [{after_min:.2f}, {after_max:

print(f"\n Total features with outlier treatment: {outlier_count}")
```

```
AmountOfVehicleDamage: Outliers capped [6575.00, 75223.00]
AmountOfInjuryClaim: Outliers capped [0.00, 19213.50]
AmountOfPropertyClaim: Outliers capped [0.00, 18239.50]
AmountOfTotalClaim: Outliers capped [8854.50, 104866.50]

Total features with outlier treatment: 4
```

Applying outlier capping to claim amount features to enhance the model.

In [27]:
```
claim_features = ['AmountOfVehicleDamage', 'AmountOfInjuryClaim',
                  'AmountOfPropertyClaim', 'AmountOfTotalClaim']

outlier_count = 0
for col in claim_features:
    if col in numerical_features:
        before_min = X_train_num_imputed[col].min()
        before_max = X_train_num_imputed[col].max()

        X_train_num_imputed = cap_outliers(X_train_num_imputed, col)
        X_test_num_imputed = cap_outliers(X_test_num_imputed, col)

        after_min = X_train_num_imputed[col].min()
        after_max = X_train_num_imputed[col].max()

        if before_min != after_min or before_max != after_max:
            outlier_count += 1
            print(f" {col}: Outliers capped [{after_min:.2f}, {after_max:

print(f"\n Total features with outlier treatment: {outlier_count}")
```

```
Total features with outlier treatment: 0
```

Categorical and numerical encoding

Label encode binary features

In [28]:
```
binary_features = []
for col in categorical_features:
    if X_train_cat_imputed[col].nunique() == 2:
        binary_features.append(col)

le = LabelEncoder()
for col in binary_features:
    X_train_cat_imputed[col] = le.fit_transform(X_train_cat_imputed[col])
    X_test_cat_imputed[col] = le.transform(X_test_cat_imputed[col])

print(f"\n Binary features label encoded: {len(binary_features)}")
```

```
Binary features label encoded: 3
```

One-hot encode remaining categorical features

In [29]:
```
onehot_features = [col for col in categorical_features if col not in bina
X_train_cat_encoded = pd.get_dummies(X_train_cat_imputed, columns=onehot_
X_test_cat_encoded = pd.get_dummies(X_test_cat_imputed, columns=onehot_fe
```

Align test set with train set columns

In [30]:
```
X_test_cat_encoded = X_test_cat_encoded.reindex(columns=X_train_cat_encod

print(f" Multi-class features one-hot encoded: {len(onehot_features)}")
print(f" Total encoded columns: {X_train_cat_encoded.shape[1]}")
```

```
Multi-class features one-hot encoded: 15
Total encoded columns: 5244
```

Combine numerical and categorical features

In [31]:
```
X_train_processed = pd.concat([X_train_num_imputed, X_train_cat_encoded],
X_test_processed = pd.concat([X_test_num_imputed, X_test_cat_encoded], ax

print(f"\n Combined dataset shape - Train: {X_train_processed.shape}, Tes
```

```
Combined dataset shape - Train: (20185, 5279), Test: (8651, 5279)
```

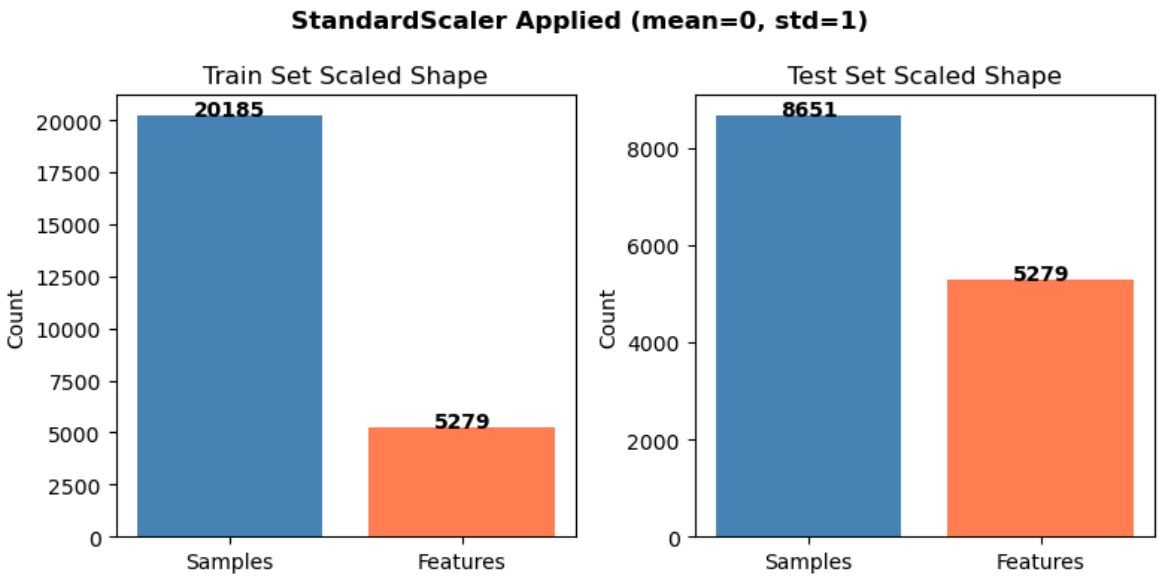
Scaling, without leaking

Applying Standard Scaler

In [32]:
```
scaler = StandardScaler()
X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train_processed),
    columns=X_train_processed.columns,
    index=X_train_processed.index
)
X_test_scaled = pd.DataFrame(
    scaler.transform(X_test_processed),
    columns=X_test_processed.columns,
    index=X_test_processed.index
)
```

In [33]:
```
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
for ax, (data, title) in zip(axes, [(X_train_scaled, 'Train'), (X_test_sc
    ax.bar(['Samples', 'Features'], data.shape, color=['steelblue', 'cora
    ax.set_title(f'{title} Set Scaled Shape')
    ax.set_ylabel('Count')
    for i, v in enumerate(data.shape):
        ax.text(i, v + 0.5, str(v), ha='center', fontweight='bold')
fig.suptitle('StandardScaler Applied (mean=0, std=1)', fontsize=12, fontw
plt.tight_layout()
plt.show()

print(f"\n StandardScaler applied | Train: {X_train_scaled.shape} | Test:
```




```
StandardScaler applied | Train: (20185, 5279) | Test: (8651, 5279)
```

Variance analysis

Check variance of features

In [34]:
```
feature_variance = X_train_scaled.var().sort_values()
low_variance_features = feature_variance[feature_variance < 0.01]

print(f"\n Features analyzed: {len(feature_variance)}")
print(f" Low variance features (<0.01): {len(low_variance_features)}")

if len(low_variance_features) > 0:
    print(f"\nLow variance features to consider removing:")
    print(low_variance_features)
```

```
Features analyzed: 5279
Low variance features (<0.01): 1

Low variance features to consider removing:
SeverityNumeric    0.0
dtype: float64
```

Apply variance threshold

In [35]:
```
var_threshold = VarianceThreshold(threshold=0.01)
var_threshold.fit(X_train_scaled)

high_var_features = X_train_scaled.columns[var_threshold.get_support()]
X_train_var_filtered = X_train_scaled[high_var_features]
X_test_var_filtered = X_test_scaled[high_var_features]

print(f"\n Features after variance filtering: {len(high_var_features)}")
print(f" Features removed: {X_train_scaled.shape[1] - len(high_var_featur
```

```
Features after variance filtering: 5278
Features removed: 1
```

Dealing with correlation and collinearity

Calculate correlation matrix

In [36]:
```
selector = SelectKBest(f_classif, k=100)  # Keep top 100 features
X_train_reduced = selector.fit_transform(X_train_var_filtered, y_train)
X_train_reduced = pd.DataFrame(X_train_reduced, columns=X_train_var_filte

# Then compute correlation on smaller set
corr_matrix = X_train_reduced.corr().abs()
```

Find highly correlated features

In [37]:
```
upper_triangle = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=
high_corr_features = [column for column in upper_triangle.columns if any(

print(f"\n Highly correlated feature pairs found (>0.90): {len(high_corr_
```

```
Highly correlated feature pairs found (>0.90): 4
['AmountOfVehicleDamage', 'ClaimToPremiumRatio', 'SeverityXClaim__missin
g', 'SeverityOfIncident_Total Loss']
```

Remove highly correlated features

In [38]:
```
X_train_final = X_train_var_filtered.drop(columns=high_corr_features, err
X_test_final = X_test_var_filtered.drop(columns=high_corr_features, error

print(f" Features after correlation removal: {X_train_final.shape[1]}")
print(f" Final feature set - Train: {X_train_final.shape}, Test: {X_test_
```

```
Features after correlation removal: 5274
Final feature set - Train: (20185, 5274), Test: (8651, 5274)
```

Aggressive Feature reduction

In [39]:
```
# Top-K most statistically relevant features using ANOVA F-tests and forc
TOP_K = 100
topk_selector = SelectKBest(score_func=f_classif, k=TOP_K)
X_train_topk_array = topk_selector.fit_transform(X_train_final, y_train)
X_test_topk_array = topk_selector.transform(X_test_final)

selected_topk_features = X_train_final.columns[topk_selector.get_support(

X_train_topk = pd.DataFrame(X_train_topk_array, columns=selected_topk_fea
X_test_topk = pd.DataFrame(X_test_topk_array, columns=selected_topk_featu

# Overwrite the feature matrices used downstream so all models train on T
X_train_final = X_train_topk
X_test_final = X_test_topk

print(f"\n Top-K feature reduction applied")
print(f" Selected features: {len(selected_topk_features)}")
print(f" Train shape: {X_train_final.shape} | Test shape: {X_test_final.s
print(f" Sample selected features: {list(selected_topk_features[:10])}")
```

```
Top-K feature reduction applied
Selected features: 100
Train shape: (20185, 100) | Test shape: (8651, 100)
Sample selected features: ['CustomerLoyaltyPeriod', 'PolicyAnnualPremiu
m', 'UmbrellaLimit', 'NumberOfVehicles', 'BodilyInjuries', 'Witnesses', 'A
mountOfTotalClaim', 'AmountOfInjuryClaim', 'AmountOfPropertyClaim', 'Claim
Delay']
```

Identifying and dealing with class imbalance

In [40]:
```
class_counts = y_train.value_counts()
imbalance_ratio = class_counts[0] / class_counts[1]

print(f"\n Imbalance ratio (Legitimate:Fraud): {imbalance_ratio:.2f}:1")
print(f" Minority class percentage: {(class_counts[1]/len(y_train))*100:.

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Bar chart
axes[0].bar(['Legitimate (0)', 'Fraud (1)'], class_counts.values, color=[
axes[0].set_title('Class Distribution in Training Set')
axes[0].set_ylabel('Count')
for i, v in enumerate(class_counts.values):
    axes[0].text(i, v + 0.01 * max(class_counts.values), str(v), ha='cent

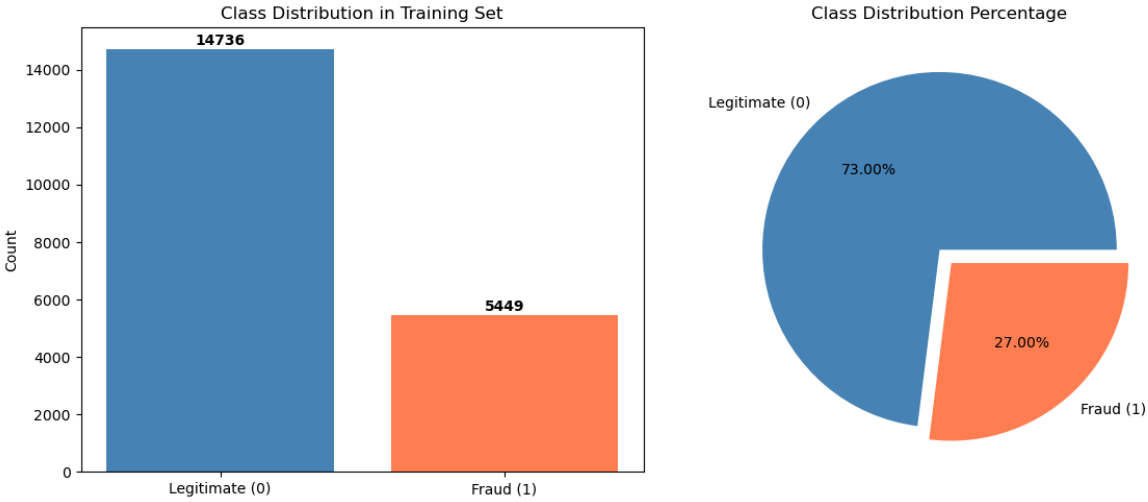
# Pie chart
axes[1].pie(class_counts.values, labels=['Legitimate (0)', 'Fraud (1)'],
            autopct='%1.2f%%', colors=['steelblue', 'coral'], explode=[0,
```

```
axes[1].set_title('Class Distribution Percentage')

plt.tight_layout()
plt.show()


if imbalance_ratio > 1.5:
    print(f"\nSignificant imbalance detected - will address during model
```

```
Imbalance ratio (Legitimate:Fraud): 2.70:1
Minority class percentage: 27.00%
```




```
Significant imbalance detected - will address during model training
```

Other visualisations

Exploratory Data Analysis via Visual Analytics for Fraud Detection

In [41]:
```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

# 1. Distribution of numerical features (sample)
numerical_sample = X_train_final.iloc[:, :3]
for idx, col in enumerate(numerical_sample.columns):
    if idx < 3:
        axes[0, idx].hist(numerical_sample[col], bins=30, color='steelblu
        axes[0, idx].set_title(f'Distribution: {col}')
        axes[0, idx].set_xlabel('Value')
        axes[0, idx].set_ylabel('Frequency')
        axes[0, idx].grid(True, alpha=0.3)

# 2. Fraud vs Non-Fraud distribution for key claim features
claim_cols = ['AmountOfVehicleDamage', 'AmountOfInjuryClaim', 'AmountOfTo
claim_data = []
for col in claim_cols:
    if col in X_train_final.columns:
        claim_data.append(X_train_final[col])

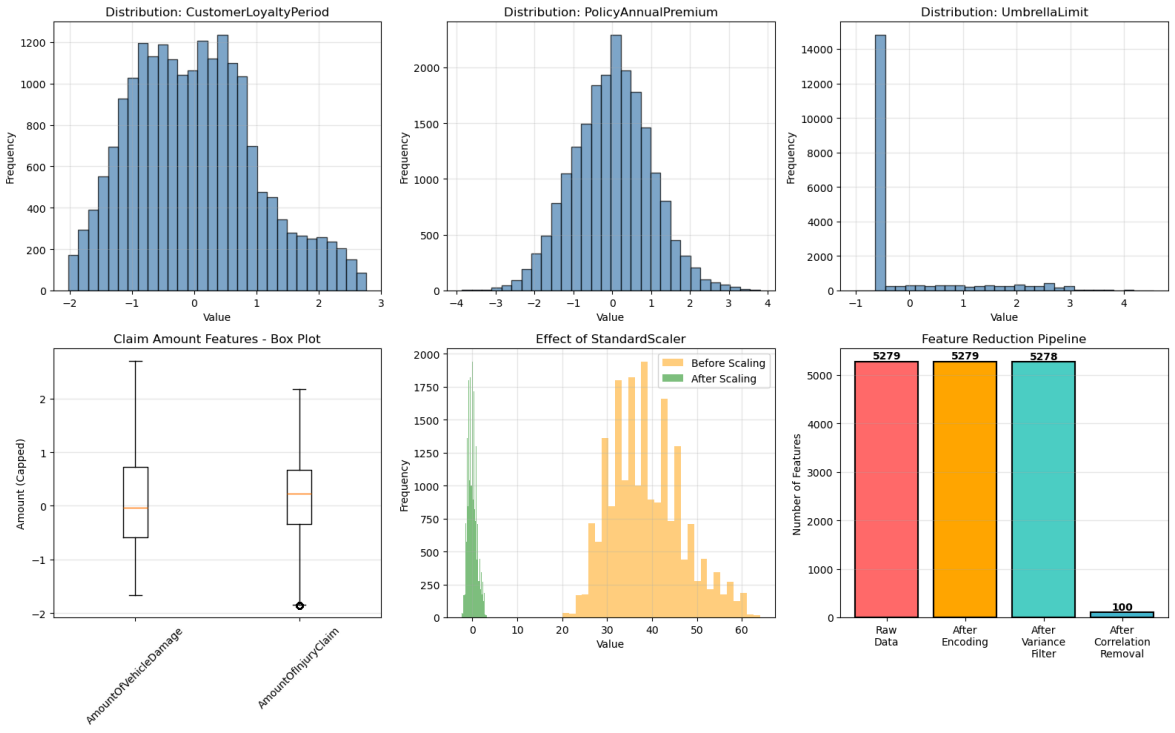
if len(claim_data) > 0:
    axes[1, 0].boxplot(claim_data, labels=claim_cols[:len(claim_data)])
    axes[1, 0].set_title('Claim Amount Features - Box Plot')
    axes[1, 0].set_ylabel('Amount (Capped)')
    axes[1, 0].tick_params(axis='x', rotation=45)
    axes[1, 0].grid(True, alpha=0.3, axis='y')

# 3. Feature scaling effect (before vs after)
```

```
sample_feature = X_train_processed.iloc[:, 0] if X_train_processed.shape[
if sample_feature is not None:
    axes[1, 1].hist(X_train_processed.iloc[:, 0], bins=30, alpha=0.5, lab
    axes[1, 1].hist(X_train_scaled.iloc[:, 0], bins=30, alpha=0.5, label=
    axes[1, 1].set_title('Effect of StandardScaler')
    axes[1, 1].set_xlabel('Value')
    axes[1, 1].set_ylabel('Frequency')
    axes[1, 1].legend()
    axes[1, 1].grid(True, alpha=0.3)

# 4. Feature count reduction through preprocessing
stages = ['Raw\nData', 'After\nEncoding', 'After\nVariance\nFilter', 'Aft
feature_counts = [X_train_processed.shape[1], X_train_scaled.shape[1], X_
colors_bar = ['#FF6B6B', '#FFA500', '#4ECDC4', '#45B7D1']
axes[1, 2].bar(stages, feature_counts, color=colors_bar, edgecolor='black
axes[1, 2].set_title('Feature Reduction Pipeline')
axes[1, 2].set_ylabel('Number of Features')
for i, v in enumerate(feature_counts):
    axes[1, 2].text(i, v + 50, str(v), ha='center', fontweight='bold')
axes[1, 2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



- Outliers handled
- Features standardized
- Skewed variables identified
- Dimensionality reduced from ~5,300 → 100
- Data ready for robust classification models

4. Technique 1: LOGISTIC REGRESSION

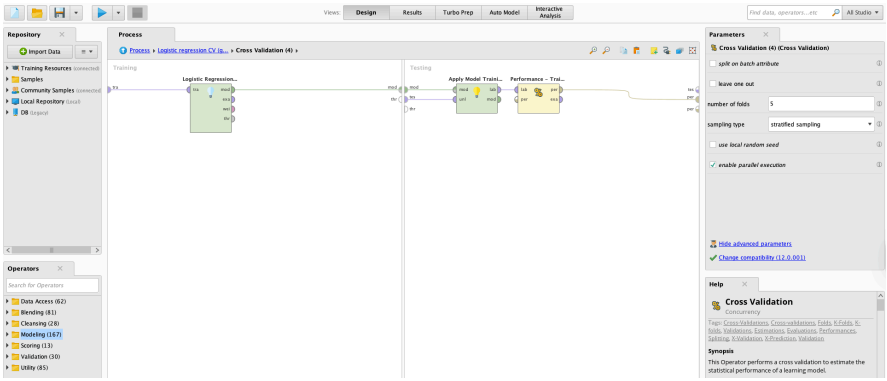
==========

Motivation for choosing the technique and schematic figure of the analysis process (100-200 words)

Logistic Regression is selected as the initial method because it provides the best balance between interpretability and suitability for the binary fraud classification problem.

- **Direct Fraud/Not Fraud outcome:** Unlike Linear Regression models (which predict continuous values) or Unsupervised models (which lack a known target label), Logistic Regression directly handles the "Fraud/Not Fraud" outcome.
- **Initial insights:** The resulting coefficients provide the client with justifiable, data-driven insights into which features increase the risk of fraud, a vital requirement for trust and business adoption.
- **Baseline performance:** Establishes a strong linear baseline to compare against more complex non-linear models.
- **Regularization:** Built-in L1/L2 regularization controls overfitting and performs implicit feature selection.
- **Interpretability:** Provides clear coefficient values showing the linear impact of each feature on fraud probability, essential for business stakeholders.

Visualize preprocessing workflow



Initial Hyperparameters Setting (Rationale)

In [42]:
```
lr_initial = LogisticRegression(
    C=1.0,                    # Regularization strength
    penalty='l2',             # Regularization penalty. 'l1' is Lasso, 'l
    class_weight='balanced',  # Weighted class. 'balanced' gives equal we
    solver='lbfgs',           # Optimization algorithm. 'lbfgs' is an eff
```

```
    max_iter=1000,              # Maximum number of iterations
    random_state=42,            # Random seed for reproducibility
    n_jobs=-1                   # Number of CPU cores to use. -1 means usin
)
```

Optimizing Hyperparameters

Grid Search Cross Validation

In [43]:
```
param_grid_lr = {
    'C': [0.01],                    # [0.01, 0.1, 1.0, 10.0, 100.0],
    'penalty': ['l2'],              # ['l1', 'l2'],
    'solver': ['liblinear']         # ['liblinear', 'saga']
}

cv_strategy = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

grid_search_lr = GridSearchCV(
    estimator=LogisticRegression(class_weight='balanced', max_iter=1000,
    param_grid=param_grid_lr,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=1
)

print("\nPerforming Grid Search CV...")
print(f"Parameter grid: {param_grid_lr}")
print(f"CV Strategy: 5-Fold Stratified Cross-Validation")
print(f"Scoring Metric: Balanced Accuracy\n")

grid_search_lr.fit(X_train_final, y_train)

print(f"\nBest parameters: {grid_search_lr.best_params_}")
print(f"Best CV balanced accuracy: {grid_search_lr.best_score_:.4f}")
```

```
Performing Grid Search CV...
Parameter grid: {'C': [0.01], 'penalty': ['l2'], 'solver': ['liblinear']}
CV Strategy: 5-Fold Stratified Cross-Validation
Scoring Metric: Balanced Accuracy

Fitting 3 folds for each of 1 candidates, totalling 3 fits

Best parameters: {'C': 0.01, 'penalty': 'l2', 'solver': 'liblinear'}
Best CV balanced accuracy: 0.7968
```

Use best model

In [44]:
```
lr_model = grid_search_lr.best_estimator_
```

Appropriate Feature Selection - Recursive Feature Elimination(RFE)

Use RFE with optimized logistic regression

In [45]:
```
rfe_selector = RFE(
    estimator=lr_model,
```

```
    n_features_to_select=30,
    step=0.5
)
rfe_selector.fit(X_train_final, y_train)

selected_features_lr = X_train_final.columns[rfe_selector.support_]
X_train_rfe_lr = X_train_final[selected_features_lr]
X_test_rfe_lr = X_test_final[selected_features_lr]

print(f" Features selected by RFE: {len(selected_features_lr)}")
print(f" Top 10 selected features: {list(selected_features_lr[:10])}")
```

```
Features selected by RFE: 30
Top 10 selected features: ['UmbrellaLimit', 'Witnesses', 'AmountOfTotalCl
aim', 'AmountOfPropertyClaim', 'ClaimDelay', 'InjuryToTotalRatio', 'Author
itiesContacted__missing', 'SeverityNumeric__missing', 'InsuredEducationLev
el_PhD', 'InsuredOccupation_exec-managerial']
```

Retrain on selected features

In [46]:
```
lr_model_final = LogisticRegression(**grid_search_lr.best_params_,
                                    class_weight='balanced',
                                    max_iter=1000,
                                    random_state=42)
lr_model_final.fit(X_train_rfe_lr, y_train)
```

Out[46]:

LogisticRegression

```
LogisticRegression(C=0.01, class_weight='balanced', max_iter=100
0,
                   random_state=42, solver='liblinear')
```

Performance Metrics for Training Set

Training Set Performance (Logistic Regression)

In [47]:
```
y_train_pred_lr = lr_model_final.predict(X_train_rfe_lr)
y_train_proba_lr = lr_model_final.predict_proba(X_train_rfe_lr)[:, 1]

train_accuracy_lr = lr_model_final.score(X_train_rfe_lr, y_train)
train_balanced_acc_lr = balanced_accuracy_score(y_train, y_train_pred_lr)
train_precision_lr = precision_score(y_train, y_train_pred_lr)
train_recall_lr = recall_score(y_train, y_train_pred_lr)
train_f1_lr = f1_score(y_train, y_train_pred_lr)
train_roc_auc_lr = roc_auc_score(y_train, y_train_proba_lr)

print(f"\n  Accuracy:          {train_accuracy_lr:.4f}")
print(f"  Balanced Accuracy: {train_balanced_acc_lr:.4f}")
print(f"  Precision:         {train_precision_lr:.4f}")
print(f"  Recall:            {train_recall_lr:.4f}")
print(f"  F1-Score:          {train_f1_lr:.4f}")
print(f"  ROC-AUC:           {train_roc_auc_lr:.4f}")
```

```
Accuracy:          0.8147
Balanced Accuracy: 0.7953
Precision:         0.6315
Recall:            0.7532
F1-Score:          0.6870
ROC-AUC:           0.8460
```

Training confusion matrix

In [48]:
```
cm_train_lr = confusion_matrix(y_train, y_train_pred_lr)
print(cm_train_lr)
```

```
[[12341  2395]
 [ 1345  4104]]
```

Other Items: Coefficient Analysis

coefficient analysis (interpretability)

In [49]:
```
coefficients_lr = pd.DataFrame({
    'Feature': selected_features_lr,
    'Coefficient': lr_model_final.coef_[0]
}).sort_values('Coefficient', ascending=False, key=abs)
```

Top 10 Most Important Features (by absolute coefficient)

In [50]:
```
print(coefficients_lr.head(10).to_string(index=False))
```

```
                         Feature  Coefficient
 SeverityOfIncident_Minor Damage    -1.018402
         SeverityNumeric__missing     0.980100
SeverityOfIncident_Trivial Damage    -0.596569
             InsuredHobbies_chess     0.541407
         InsuredHobbies_cross-fit     0.502056
               AmountOfTotalClaim     0.250033
            IncidentState_State9    -0.187552
          InsuredHobbies_dancing    -0.171270
            IncidentState_State5    -0.157335
              InjuryToTotalRatio    -0.149245
```

Top 10 Features Increasing Fraud Probability (positive coefficients)

In [51]:
```
print(coefficients_lr[coefficients_lr['Coefficient'] > 0].head(10).to_str
```

```
                         Feature  Coefficient
         SeverityNumeric__missing     0.980100
             InsuredHobbies_chess     0.541407
         InsuredHobbies_cross-fit     0.502056
               AmountOfTotalClaim     0.250033
InsuredOccupation_exec-managerial     0.145093
                     UmbrellaLimit     0.135685
                         Witnesses     0.122224
    InsuredRelationship_unmarried     0.118012
  TypeOfCollission_Rear Collision     0.112925
InsuredRelationship_other-relative     0.100988
```

Top 10 Features Decreasing Fraud Probability (negative coefficients)

In [52]:
```
print(coefficients_lr[coefficients_lr['Coefficient'] < 0].tail(10).to_str
```

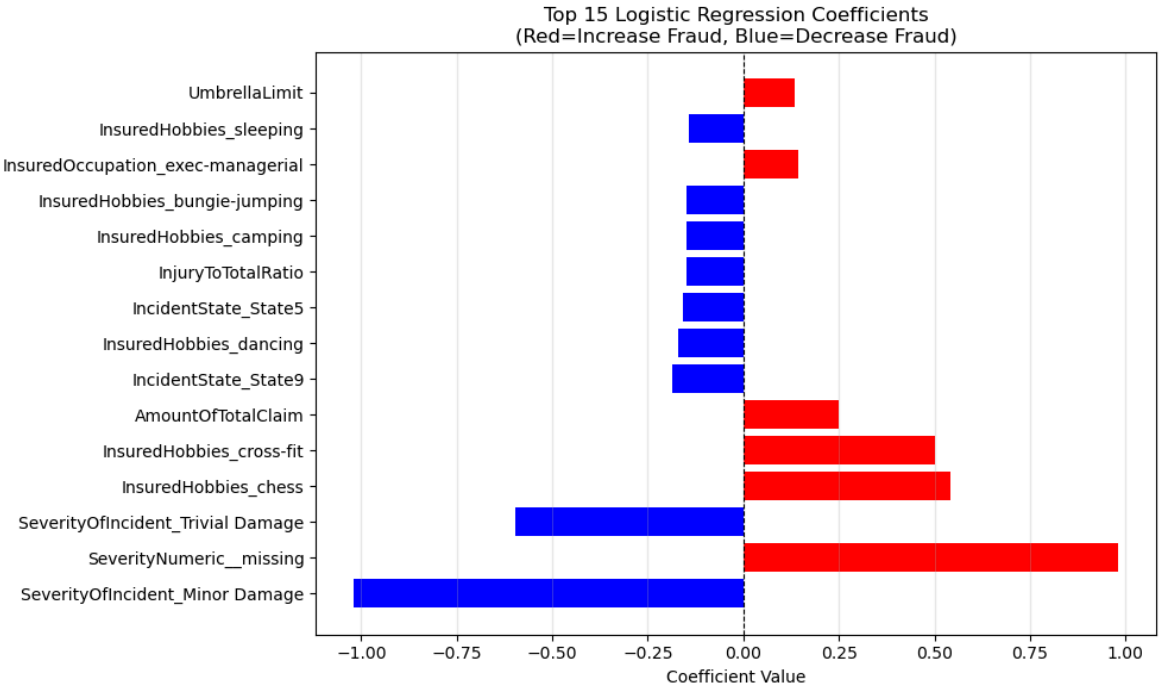
Feature	Coefficient
InsuredHobbies_sleeping	-0.141106
AuthoritiesContacted__missing	-0.119005
VehicleAttributeDetails_Wrangler	-0.117116
InsuredHobbies_basketball	-0.105089
InsuredHobbies_golf	-0.102932
IncidentState_State8	-0.101852
InsuredHobbies_kayaking	-0.090139
AmountOfPropertyClaim	-0.088378
InsuredHobbies_exercise	-0.087744
InsuredOccupation_handlers-cleaners	-0.078479

Visualize coefficients

In [53]:
```
fig, ax = plt.subplots(figsize=(10, 6))
top_coef = coefficients_lr.head(15)
colors = ['red' if c > 0 else 'blue' for c in top_coef['Coefficient']]

ax.barh(range(len(top_coef)), top_coef['Coefficient'], color=colors)
ax.set_yticks(range(len(top_coef)))
ax.set_yticklabels(top_coef['Feature'])
ax.set_xlabel('Coefficient Value')
ax.set_title('Top 15 Logistic Regression Coefficients\n(Red=Increase Frau
ax.axvline(x=0, color='black', linestyle='--', linewidth=0.8)
ax.grid(True, alpha=0.3, axis='x')

plt.tight_layout()
plt.show()
```



Here is a simple, non-technical bullet explanation of the chart:

- 15 most important features used by the fraud detection model.
- Red bars mean the feature increases the chance of fraud.
- Blue bars indicate the feature decreases the chance of fraud.
- Longer bars mean the feature has a stronger effect.

Features that increase fraud risk (Red):

- Missing severity information → very strong fraud signal.
- Higher total claim amount → fraud becomes more likely.
- High umbrella insurance limit → slightly higher fraud risk.
- Certain hobbies (chess, cross-fit) → linked to higher fraud probability.
- Executive/managerial occupation → higher fraud association.

Features that reduce fraud risk (Blue):

- Minor or trivial damage → very unlikely to be fraud.
- Balanced injury vs total claim amount → looks legitimate.
- Certain states → historically lower fraud cases.
- Some hobbies (sleeping, camping, dancing, bungee-jumping) → lower fraud association.

Key takeaway:

- Big claims + missing information = high fraud risk.
- Small damage + consistent details = low fraud risk.

==========

5. Technique 2: RANDOM FOREST

==========

Motivation for choosing the technique and schematic figure of the analysis process (100-200 words)

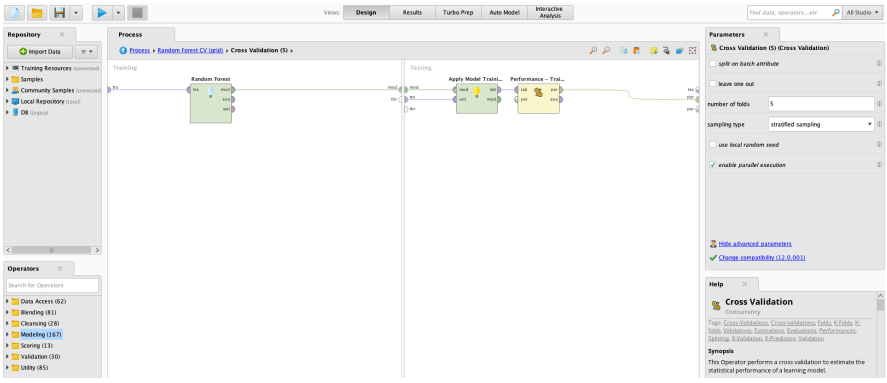
Random Forest is chosen as the second, more advanced technique to complement Logistic Regression, primarily for its high accuracy and robustness in complex, non-linear fraud detection scenarios. Below are additional key reasons:

1. **Non-linear patterns**: Captures complex non-linear relationships and feature interactions that linear models miss.
2. **Robustness**: Ensemble of decision trees reduces overfitting through averaging and handles outliers effectively.
3. **Feature importance**: Provides intuitive importance scores based on impurity reduction, complementing linear coefficients.
4. **No scaling required**: Tree-based methods are invariant to feature scaling, though we maintain consistency with preprocessing.

5. **Handles mixed data**: Naturally processes both numerical and categorical features without extensive preprocessing.

6. **Reduced variance**: Bagging reduces model variance through bootstrap sampling and random feature selection.

Random Forest Workflow



Setting Hyperparameters (Rationale)

Initial model

In [54]:
```
rf_initial = RandomForestClassifier(

    n_estimators=200,          # Number of trees in the forest
    max_depth=20,              # Maximum depth of each tree
    min_samples_split=10,      # Minimum number of samples to split a node
    min_samples_leaf=5,        # Minimum number of samples at each leaf no
    class_weight='balanced',   # Weights classes to balance classes if cla
    max_features='sqrt',       # Number of features to consider when looki
    random_state=42,           # Random state for reproducibility
    n_jobs=-1                  # Number of CPU cores to use for parallel e
)
```

Optimizing Hyperparameters

Grid search cv

In [55]:
```
param_grid_rf = {
    'n_estimators': [200, 300, 400],
    'max_depth': [5, 10, 15],
    'min_samples_split': [20, 30, 40],
    'min_samples_leaf': [10, 15, 20],
    'max_features': ['sqrt', 'log2']
}

grid_search_rf = GridSearchCV(
    estimator=RandomForestClassifier(class_weight='balanced', random_stat
    param_grid=param_grid_rf,
    cv=cv_strategy,
    scoring='balanced_accuracy',
    n_jobs=-1,
    verbose=1
```

```
)

print("\nPerforming Grid Search CV...")
print(f"Parameter grid: {param_grid_rf}")
print(f"CV Strategy: 3-Fold Stratified Cross-Validation")
print(f"Scoring Metric: Balanced Accuracy\n")

grid_search_rf.fit(X_train_final, y_train)

print(f"\nBest parameters: {grid_search_rf.best_params_}")
print(f"Best CV balanced accuracy: {grid_search_rf.best_score_:.4f}")
```

```
Performing Grid Search CV...
Parameter grid: {'n_estimators': [200, 300, 400], 'max_depth': [5, 10, 1
5], 'min_samples_split': [20, 30, 40], 'min_samples_leaf': [10, 15, 20],
'max_features': ['sqrt', 'log2']}
CV Strategy: 3-Fold Stratified Cross-Validation
Scoring Metric: Balanced Accuracy

Fitting 3 folds for each of 162 candidates, totalling 486 fits

Best parameters: {'max_depth': 15, 'max_features': 'log2', 'min_samples_le
af': 10, 'min_samples_split': 20, 'n_estimators': 400}
Best CV balanced accuracy: 0.8334
```

Use best model

In [56]:
```
rf_model = grid_search_rf.best_estimator_
```

Appropriate Feature Selection

Get feature importances

In [57]:
```
feature_importances_rf = pd.DataFrame({
    'Feature': X_train_final.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

print(f"\nTop 15 Most Important Features (Random Forest):")
print(feature_importances_rf.head(15).to_string(index=False))
```

```
Top 15 Most Important Features (Random Forest):
                      Feature  Importance
SeverityOfIncident_Minor Damage    0.112163
       SeverityNumeric__missing    0.080620
           InsuredHobbies_chess    0.063269
       InsuredHobbies_cross-fit    0.056823
                  SeverityXClaim    0.050239
          AmountOfPropertyClaim    0.048462
             AmountOfTotalClaim    0.045506
            AmountOfInjuryClaim    0.041972
             InjuryToTotalRatio    0.041596
                     ClaimDelay    0.036320
            PolicyAnnualPremium    0.031716
           CustomerLoyaltyPeriod    0.029974
                   UmbrellaLimit    0.024089
     TypeOfCollission__missing    0.018829
                     Witnesses    0.017947
```

Select top 30 important features

In [58]:
```
top_n_features = 30
selected_features_rf = feature_importances_rf.head(top_n_features)['Featu
X_train_selected_rf = X_train_final[selected_features_rf]
X_test_selected_rf = X_test_final[selected_features_rf]
```

Retrain Model on selected features

In [59]:
```
rf_model_final = RandomForestClassifier(**grid_search_rf.best_params_,
                                        class_weight='balanced',
                                        random_state=42,
                                        n_jobs=-1)
rf_model_final.fit(X_train_selected_rf, y_train);
```

Performance Metrics

Training Set Performance (Random Forest)

In [60]:
```
y_train_pred_rf = rf_model_final.predict(X_train_selected_rf)
y_train_proba_rf = rf_model_final.predict_proba(X_train_selected_rf)[:, 1

train_accuracy_rf = rf_model_final.score(X_train_selected_rf, y_train)
train_balanced_acc_rf = balanced_accuracy_score(y_train, y_train_pred_rf)
train_precision_rf = precision_score(y_train, y_train_pred_rf)
train_recall_rf = recall_score(y_train, y_train_pred_rf)
train_f1_rf = f1_score(y_train, y_train_pred_rf)
train_roc_auc_rf = roc_auc_score(y_train, y_train_proba_rf)

print(f"  Accuracy:          {train_accuracy_rf:.4f}")
print(f"  Balanced Accuracy: {train_balanced_acc_rf:.4f}")
print(f"  Precision:         {train_precision_rf:.4f}")
print(f"  Recall:            {train_recall_rf:.4f}")
print(f"  F1-Score:          {train_f1_rf:.4f}")
print(f"  ROC-AUC:           {train_roc_auc_rf:.4f}")
```

```
 Accuracy:          0.8776
 Balanced Accuracy: 0.8621
 Precision:         0.7461
 Recall:            0.8284
 F1-Score:          0.7851
 ROC-AUC:           0.9575
```

Training confusion matrix

In [61]:
```
cm_train_rf = confusion_matrix(y_train, y_train_pred_rf)
print(cm_train_rf)
```

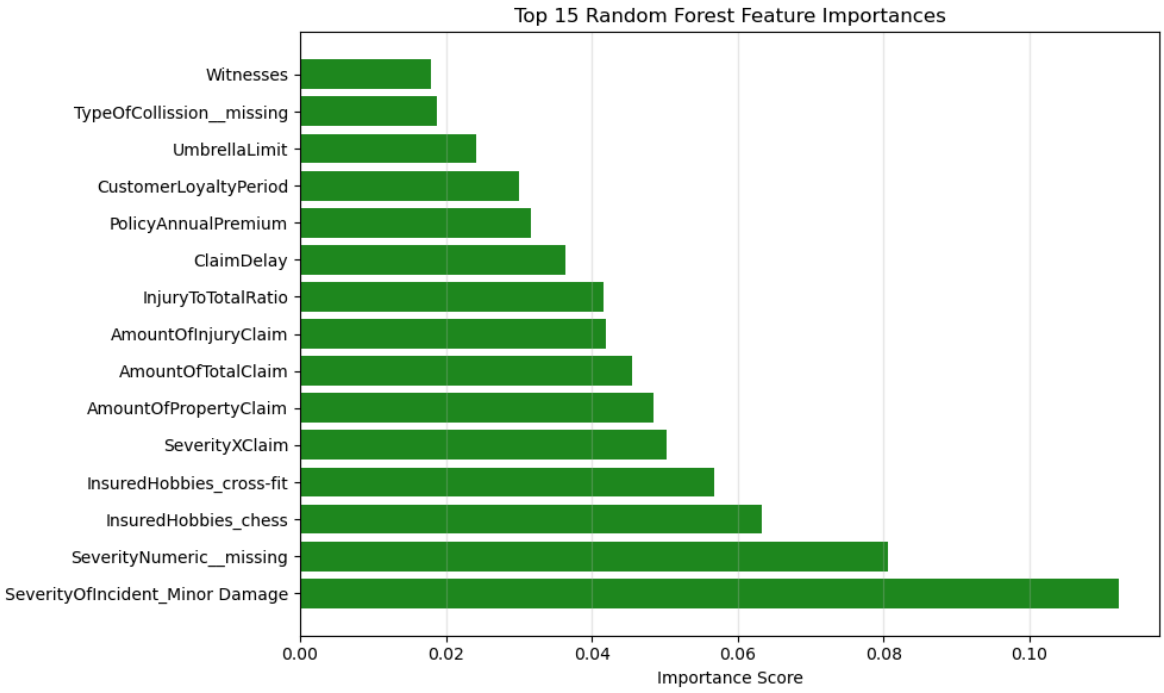
```
[[13200  1536]
 [  935  4514]]
```

Feature importance visualization

In [62]:
```
fig, ax = plt.subplots(figsize=(10, 6))
top_features = feature_importances_rf.head(15)
```

```
ax.barh(range(len(top_features)), top_features['Importance'], color='fore
ax.set_yticks(range(len(top_features)))
ax.set_yticklabels(top_features['Feature'])
ax.set_xlabel('Importance Score')
ax.set_title('Top 15 Random Forest Feature Importances')
ax.grid(True, alpha=0.3, axis='x')

plt.tight_layout()
plt.show()
```



Random Forest Feature Importance

- The chart shows the 15 most important features used by the Random Forest model.
- Longer bars = more important for the model's decisions.

Most Important Signals

- Minor damage severity → strongest signal in the model
- Missing severity information → very informative for detecting fraud

Policy & Customer Features

- Annual premium
- Umbrella coverage limit
- Customer loyalty period

Policy value and customer history influence predictions.

Data Quality Features

- Missing collision type
- Missing severity values

Missing data itself is a strong signal.

Least Important (Among Top 15)

- Number of witnesses

Key Takeaway

- Missing information is a red flag

==========

6. Comparison of metrics performance for testing

==========

Nested Cross-Validation

Nested cross-validation provides unbiased performance estimates by:

1. OUTER LOOP: Splits data for model assessment (5 folds)
2. INNER LOOP: Splits training data for hyperparameter tuning (3 folds)

Nested CV for Logistic Regression

In [63]:
```
outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

nested_scores_lr = []

for train_idx, val_idx in outer_cv.split(X_train_rfe_lr, y_train):
    X_train_cv, X_val_cv = X_train_rfe_lr.iloc[train_idx], X_train_rfe_lr
    y_train_cv, y_val_cv = y_train.iloc[train_idx], y_train.iloc[val_idx]

    # Inner CV for hyperparameter tuning
    inner_grid = GridSearchCV(
        LogisticRegression(class_weight='balanced', max_iter=1000, random
        param_grid={'C': [0.1, 1.0, 10.0], 'penalty': ['l2']},
        cv=inner_cv,
        scoring='balanced_accuracy'
    )
    inner_grid.fit(X_train_cv, y_train_cv)

    # Evaluate on outer fold
    y_val_pred = inner_grid.predict(X_val_cv)
    score = balanced_accuracy_score(y_val_cv, y_val_pred)
    nested_scores_lr.append(score)

print(f" Nested CV Balanced Accuracy (LR): {np.mean(nested_scores_lr):.4f
```

```
Nested CV Balanced Accuracy (LR): 0.7944 (+/- 0.0030)
```

Nested CV for Random Forest

In [64]:
```
nested_scores_rf = []

for train_idx, val_idx in outer_cv.split(X_train_selected_rf, y_train):
    X_train_cv, X_val_cv = X_train_selected_rf.iloc[train_idx], X_train_s
    y_train_cv, y_val_cv = y_train.iloc[train_idx], y_train.iloc[val_idx]

    # Inner CV for hyperparameter tuning
    inner_grid = GridSearchCV(
        RandomForestClassifier(class_weight='balanced', random_state=42,
        param_grid={'n_estimators': [100, 200], 'max_depth': [10, 20]},
        cv=inner_cv,
        scoring='balanced_accuracy'
    )
    inner_grid.fit(X_train_cv, y_train_cv)

    # Evaluate on outer fold
    y_val_pred = inner_grid.predict(X_val_cv)
    score = balanced_accuracy_score(y_val_cv, y_val_pred)
    nested_scores_rf.append(score)

print(f" Nested CV Balanced Accuracy (RF): {np.mean(nested_scores_rf):.4f
```

```
Nested CV Balanced Accuracy (RF): 0.8592 (+/- 0.0031)
```

Test Set Performance

Logistic Regression on test set

In [65]:
```
# Logistic Regression Metrics
y_test_pred_lr = lr_model_final.predict(X_test_rfe_lr)
y_test_proba_lr = lr_model_final.predict_proba(X_test_rfe_lr)[:, 1]

test_accuracy_lr = lr_model_final.score(X_test_rfe_lr, y_test)
test_balanced_acc_lr = balanced_accuracy_score(y_test, y_test_pred_lr)
test_precision_lr = precision_score(y_test, y_test_pred_lr)
test_recall_lr = recall_score(y_test, y_test_pred_lr)
test_f1_lr = f1_score(y_test, y_test_pred_lr)
test_roc_auc_lr = roc_auc_score(y_test, y_test_proba_lr)
```

Logistic Regression metrics results

In [66]:
```
lr_results = pd.DataFrame({
    "Metric": ["Accuracy", "Balanced Accuracy", "Precision", "Recall", "F
    "Value": [test_accuracy_lr, test_balanced_acc_lr, test_precision_lr,
})
lr_results
```

Out[66]:

	Metric	Value
0	Accuracy	0.810773
1	Balanced Accuracy	0.791088
2	Precision	0.624955
3	Recall	0.748288
4	F1-Score	0.681083
5	ROC-AUC	0.840524

Logistic Regression Confusion Matrix

In [67]:
```
cm_test_lr = confusion_matrix(y_test, y_test_pred_lr)
cm_test_lr
```

Out[67]:
```
array([[5266, 1049],
       [ 588, 1748]])
```

Random Forest on test set

In [68]:
```
y_test_pred_rf = rf_model_final.predict(X_test_selected_rf)
y_test_proba_rf = rf_model_final.predict_proba(X_test_selected_rf)[:, 1]

test_accuracy_rf = rf_model_final.score(X_test_selected_rf, y_test)
test_balanced_acc_rf = balanced_accuracy_score(y_test, y_test_pred_rf)
test_precision_rf = precision_score(y_test, y_test_pred_rf)
test_recall_rf = recall_score(y_test, y_test_pred_rf)
test_f1_rf = f1_score(y_test, y_test_pred_rf)
test_roc_auc_rf = roc_auc_score(y_test, y_test_proba_rf)
```

In [69]:
```
rf_results = pd.DataFrame({
    "Metric": [ "Accuracy", "Balanced Accuracy", "Precision", "Recall", "
    "Random Forest": [test_accuracy_rf,test_balanced_acc_rf,test_precisio
})

rf_results
```

Out[69]:

	Metric	Random Forest
0	Accuracy	0.853196
1	Balanced Accuracy	0.832283
2	Precision	0.704215
3	Recall	0.786815
4	F1-Score	0.743227
5	ROC-AUC	0.897717

In [70]:
```
cm_test_rf = confusion_matrix(y_test, y_test_pred_rf)
cm_test_rf
```

Out[70]:
```
array([[5543,  772],
       [ 498, 1838]])
```

Model Selection Visualization Curves

ROC Curve

In [71]:
```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# ROC Curve
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_test_proba_lr)
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_test_proba_rf)

axes[0, 0].plot(fpr_lr, tpr_lr, label=f'Logistic Regression (AUC={test_ro
axes[0, 0].plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC={test_roc_auc_
axes[0, 0].plot([0, 1], [0, 1], 'k--', label='Random Classifier')
axes[0, 0].set_xlabel('False Positive Rate')
axes[0, 0].set_ylabel('True Positive Rate')
axes[0, 0].set_title('ROC Curve Comparison')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

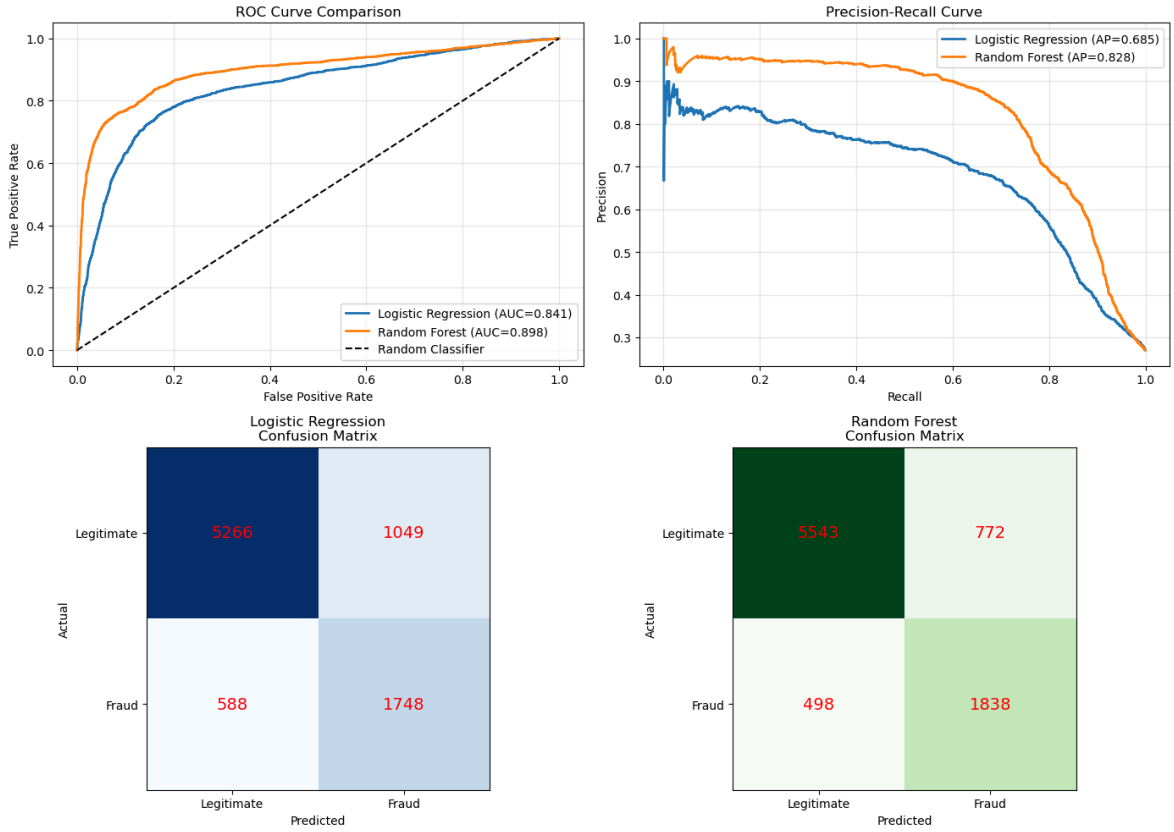
# Precision-Recall Curve
precision_lr, recall_lr, _ = precision_recall_curve(y_test, y_test_proba_
precision_rf, recall_rf, _ = precision_recall_curve(y_test, y_test_proba_
ap_lr = average_precision_score(y_test, y_test_proba_lr)
ap_rf = average_precision_score(y_test, y_test_proba_rf)

axes[0, 1].plot(recall_lr, precision_lr, label=f'Logistic Regression (AP=
axes[0, 1].plot(recall_rf, precision_rf, label=f'Random Forest (AP={ap_rf
axes[0, 1].set_xlabel('Recall')
axes[0, 1].set_ylabel('Precision')
axes[0, 1].set_title('Precision-Recall Curve')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Confusion Matrices
im1 = axes[1, 0].imshow(cm_test_lr, cmap='Blues', interpolation='nearest'
axes[1, 0].set_title('Logistic Regression\nConfusion Matrix')
axes[1, 0].set_xlabel('Predicted')
axes[1, 0].set_ylabel('Actual')
for i in range(2):
    for j in range(2):
        axes[1, 0].text(j, i, cm_test_lr[i, j], ha='center', va='center',
axes[1, 0].set_xticks([0, 1])
axes[1, 0].set_yticks([0, 1])
axes[1, 0].set_xticklabels(['Legitimate', 'Fraud'])
axes[1, 0].set_yticklabels(['Legitimate', 'Fraud'])

im2 = axes[1, 1].imshow(cm_test_rf, cmap='Greens', interpolation='nearest
axes[1, 1].set_title('Random Forest\nConfusion Matrix')
axes[1, 1].set_xlabel('Predicted')
axes[1, 1].set_ylabel('Actual')
for i in range(2):
    for j in range(2):
        axes[1, 1].text(j, i, cm_test_rf[i, j], ha='center', va='center',
axes[1, 1].set_xticks([0, 1])
axes[1, 1].set_yticks([0, 1])
axes[1, 1].set_xticklabels(['Legitimate', 'Fraud'])
axes[1, 1].set_yticklabels(['Legitimate', 'Fraud'])
```

```
plt.tight_layout()
plt.show()
```



ROC Curve Comparison (Top Left)

The ROC curve illustrates the balance between detecting fraud and avoiding false accusations.

- **Random Forest (AUC = 0.898):** Shows a strong ability to differentiate between the classes.
- **Logistic Regression (AUC = 0.841):** The lower AUC shows it is less effective at separating fraud from legitimate cases compared to the Forest.

Precision-Recall Curve (Top Right)

This curve focuses on the minority class (Fraud).

- **Average Precision (AP):** Random Forest: 0.828, Logistic Regression: 0.685.
- **Interpretation:** The RF model detects fraud more precisely with fewer false positives than LR, which tends to produce more false alarms.

Confusion Matrix Comparison (Bottom)

The confusion matrices provide the raw counts of correct and incorrect predictions.

- **Random Forest (The "Green" Matrix)**

- True Positives (Fraud caught) 1,838, It caught more fraud cases than Logistic Regression.
- True Negatives (Legitimate correctly identified): 5,543.

- **Logistic Regression (The "Blue" Matrix)**
  - True Positives (Fraud caught): 1,748.
  - False Positives (Legitimate flagged as Fraud): 1,049. This model is much more "trigger-happy," resulting in over 1,000 customers being falsely flagged for fraud.

Metric	Logistic Regression	Random Forest	Best ML
Detection Power	Good	Excellent	**Random Forest**
False Alarm Rate	Higher (1,049)	Lower (772) (Legitimate flagged as Fraud)	**Random Forest**
Fraud Coverage	1,748	1,838 (Fraud caught)	**Random Forest**

Checking for Overfitting

Logistic Regression Overfitting Analysis

In [72]:
```
def assess_overfitting_lr(train_balanced_acc_lr, test_balanced_acc_lr):
    print(f"  Training Balanced Accuracy: {train_balanced_acc_lr:.4f}")
    print(f"  Test Balanced Accuracy:     {test_balanced_acc_lr:.4f}")
    print(f"  Difference:                 {abs(train_balanced_acc_lr - te
    if abs(train_balanced_acc_lr - test_balanced_acc_lr) < 0.05:
        print("   No significant overfitting detected")
    else:
        print("  ⚠ Potential overfitting detected")

assess_overfitting_lr(train_balanced_acc_lr, test_balanced_acc_lr)
```

```
 Training Balanced Accuracy: 0.7953
 Test Balanced Accuracy:     0.7911
 Difference:                 0.0042
  No significant overfitting detected
```

Random Forest Overfitting Analysis

In [73]:
```
def assess_overfitting_rf(train_balanced_acc_rf, test_balanced_acc_rf):
    print(f"  Training Balanced Accuracy: {train_balanced_acc_rf:.4f}")
    print(f"  Test Balanced Accuracy:     {test_balanced_acc_rf:.4f}")
    print(f"  Difference:                 {abs(train_balanced_acc_rf - te
    if abs(train_balanced_acc_rf - test_balanced_acc_rf) < 0.05:
        print("   No significant overfitting detected")
    else:
        print("  ⚠ Potential overfitting detected")

assess_overfitting_rf(train_balanced_acc_rf, test_balanced_acc_rf)
```

```
Training Balanced Accuracy: 0.8621
Test Balanced Accuracy:     0.8323
Difference:                 0.0298
 No significant overfitting detected
```

Visualization of overfitting

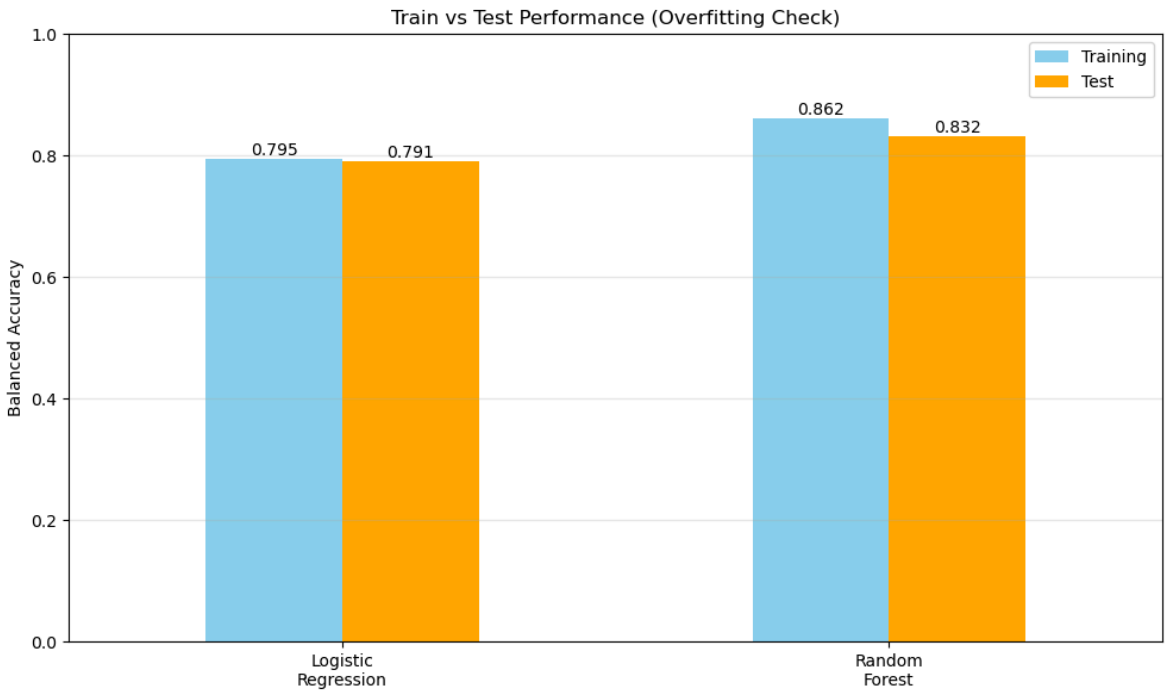
In [74]:
```
def plot_train_test_performance(train_balanced_acc_lr, train_balanced_acc
                                test_balanced_acc_lr, test_balanced_acc_r
    """Create a DataFrame for easy plotting and plot it"""
    # 1. Create a DataFrame for easy plotting
    data = pd.DataFrame({
        'Training': [train_balanced_acc_lr, train_balanced_acc_rf],
        'Test': [test_balanced_acc_lr, test_balanced_acc_rf]
    }, index=['Logistic\nRegression', 'Random\nForest'])

    # 2. Plot and label in one go
    ax = data.plot(kind='bar', figsize=(10, 6), color=['skyblue', 'orange

    # 3. Add text labels and styling
    ax.set(ylabel='Balanced Accuracy',
           title='Train vs Test Performance (Overfitting Check)',
           ylim=(0, 1))
    for p in ax.patches:
        ax.annotate(f'{p.get_height():.3f}',
                    (p.get_x() + p.get_width()/2., p.get_height()),
                    ha='center', va='bottom')

    plt.grid(True, alpha=0.3, axis='y')
    plt.tight_layout()
    plt.show()

plot_train_test_performance(train_balanced_acc_lr, train_balanced_acc_rf,
                            test_balanced_acc_lr, test_balanced_acc_rf)
```



Performance comparison summary

Model metrics comparison

In [75]:
```
def generate_comparison_df(test_balanced_acc_lr, test_precision_lr, test_
                           nested_scores_lr, train_balanced_acc_lr,
                           test_balanced_acc_rf, test_precision_rf, test_
                           nested_scores_rf, train_balanced_acc_rf):
    comparison_data = {
        'Metric': ['Balanced Accuracy', 'Precision', 'Recall', 'F1-Score'
                   'Nested CV Score', 'Train-Test Gap'],
        'Logistic Regression': [
            f"{test_balanced_acc_lr:.4f}",
            f"{test_precision_lr:.4f}",
            f"{test_recall_lr:.4f}",
            f"{test_f1_lr:.4f}",
            f"{test_roc_auc_lr:.4f}",
            f"{np.mean(nested_scores_lr):.4f} ± {np.std(nested_scores_lr)
            f"{abs(train_balanced_acc_lr - test_balanced_acc_lr):.4f}"
        ],
        'Random Forest': [
            f"{test_balanced_acc_rf:.4f}",
            f"{test_precision_rf:.4f}",
            f"{test_recall_rf:.4f}",
            f"{test_f1_rf:.4f}",
            f"{test_roc_auc_rf:.4f}",
            f"{np.mean(nested_scores_rf):.4f} ± {np.std(nested_scores_rf)
            f"{abs(train_balanced_acc_rf - test_balanced_acc_rf):.4f}"
        ]
    }

    comparison_df = pd.DataFrame(comparison_data)
    return comparison_df

comparison_df = generate_comparison_df(test_balanced_acc_lr, test_precisi
                                       nested_scores_lr, train_balanced_ac
                                       test_balanced_acc_rf, test_precisio
                                       nested_scores_rf, train_balanced_ac
comparison_df
```

Out[75]:

	Metric	Logistic Regression	Random Forest
0	Balanced Accuracy	0.7911	0.8323
1	Precision	0.6250	0.7042
2	Recall	0.7483	0.7868
3	F1-Score	0.6811	0.7432
4	ROC-AUC	0.8405	0.8977
5	Nested CV Score	0.7944 ± 0.0030	0.8592 ± 0.0031
6	Train-Test Gap	0.0042	0.0298

False positives & negatives, Fraud claims, profits, and risk assessment metrics.

In [76]:
```
fp_lr = cm_test_lr[0, 1]
fn_lr = cm_test_lr[1, 0]

fp_rf = cm_test_rf[0, 1]
```

```
fn_rf = cm_test_rf[1, 0]

fraud_claims = df[df['ReportedFraud'] == 1]['AmountOfTotalClaim']
avg_fraud_claim = fraud_claims.mean()

avg_claim = df['AmountOfTotalClaim'].mean()
gross_profit_multiplier = 2
claim_rate = 0.10
annual_policy_price = (avg_claim * gross_profit_multiplier) / claim_rate

cost_fp_rf = fp_rf * annual_policy_price
cost_fn_rf = fn_rf * avg_fraud_claim
total_cost_rf = cost_fp_rf + cost_fn_rf

cost_fp_lr = fp_lr * annual_policy_price
cost_fn_lr = fn_lr * avg_fraud_claim
total_cost_lr = cost_fp_lr + cost_fn_lr
```

Comprehensive model comparison dashboard

In [77]:

```
def create_comprehensive_dashboard():
    # Create comprehensive comparison dashboard
    fig = plt.figure(figsize=(16, 10))
    gs = fig.add_gridspec(3, 3, hspace=0.3, wspace=0.3)

    # 1. Performance Metrics Radar Chart
    ax1 = fig.add_subplot(gs[0, 0], projection='polar')
    categories = ['Balanced\nAccuracy', 'Precision', 'Recall', 'F1-Score'
    lr_values = [test_balanced_acc_lr, test_precision_lr, test_recall_lr,
    rf_values = [test_balanced_acc_rf, test_precision_rf, test_recall_rf,

    angles = np.linspace(0, 2 * np.pi, len(categories), endpoint=False).t
    lr_values += lr_values[:1]
    rf_values += rf_values[:1]
    angles += angles[:1]

    ax1.plot(angles, lr_values, 'o-', linewidth=2, label='Logistic Regres
    ax1.fill(angles, lr_values, alpha=0.25)
    ax1.plot(angles, rf_values, 'o-', linewidth=2, label='Random Forest')
    ax1.fill(angles, rf_values, alpha=0.25)
    ax1.set_xticks(angles[:-1])
    ax1.set_xticklabels(categories, size=8)
    ax1.set_ylim(0, 1)
    ax1.set_title('Performance Metrics\nComparison', size=10, pad=20)
    ax1.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
    ax1.grid(True)

    # 2. Error Analysis
    ax2 = fig.add_subplot(gs[0, 1])
    error_types = ['False\nPositives', 'False\nNegatives']
    lr_errors = [fp_lr, fn_lr]
    rf_errors = [fp_rf, fn_rf]

    x = np.arange(len(error_types))
    width = 0.35
    ax2.bar(x - width/2, lr_errors, width, label='Logistic Regression', c
    ax2.bar(x + width/2, rf_errors, width, label='Random Forest', color='
    ax2.set_ylabel('Count')
    ax2.set_title('Error Analysis')
```

```
    ax2.set_xticks(x)
    ax2.set_xticklabels(error_types)
    ax2.legend()
    ax2.grid(True, alpha=0.3, axis='y')

    # 3. ROC Comparison (zoomed)
    ax3 = fig.add_subplot(gs[0, 2])
    ax3.plot(fpr_lr, tpr_lr, label=f'LR (AUC={test_roc_auc_lr:.3f})', lin
    ax3.plot(fpr_rf, tpr_rf, label=f'RF (AUC={test_roc_auc_rf:.3f})', lin
    ax3.plot([0, 1], [0, 1], 'k--', alpha=0.5)
    ax3.set_xlabel('False Positive Rate')
    ax3.set_ylabel('True Positive Rate')
    ax3.set_title('ROC Curves')
    ax3.legend()
    ax3.grid(True, alpha=0.3)

    # 4. Class Distribution
    ax4 = fig.add_subplot(gs[1, 0])
    class_names = ['Legitimate', 'Fraud']
    train_dist = y_train.value_counts().values
    test_dist = y_test.value_counts().values

    x = np.arange(len(class_names))
    width = 0.35
    ax4.bar(x - width/2, train_dist, width, label='Train', color='lightbl
    ax4.bar(x + width/2, test_dist, width, label='Test', color='lightcora
    ax4.set_ylabel('Count')
    ax4.set_title('Class Distribution (Train vs Test)')
    ax4.set_xticks(x)
    ax4.set_xticklabels(class_names)
    ax4.legend()
    ax4.grid(True, alpha=0.3, axis='y')

    # 5. Nested CV Scores
    ax5 = fig.add_subplot(gs[1, 1])
    ax5.boxplot([nested_scores_lr, nested_scores_rf], labels=['Logistic\n
    ax5.set_ylabel('Balanced Accuracy')
    ax5.set_title('Nested CV Score Distribution')
    ax5.grid(True, alpha=0.3, axis='y')

    # 6. Cost Impact Breakdown
    ax6 = fig.add_subplot(gs[1, 2])
    cost_categories = ['FP Cost', 'FN Cost', 'Total']
    lr_costs = [cost_fp_lr, cost_fn_lr, total_cost_lr]
    rf_costs = [cost_fp_rf, cost_fn_rf, total_cost_rf]

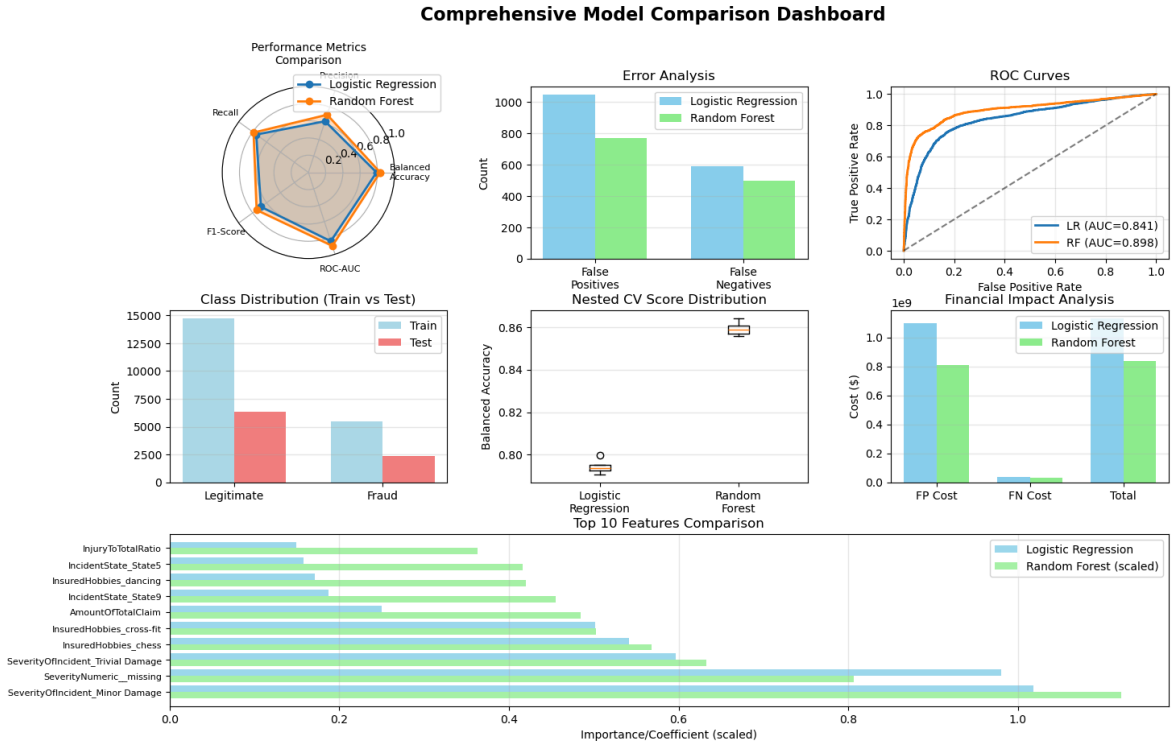
    x = np.arange(len(cost_categories))
    width = 0.35
    ax6.bar(x - width/2, lr_costs, width, label='Logistic Regression', co
    ax6.bar(x + width/2, rf_costs, width, label='Random Forest', color='l
    ax6.set_ylabel('Cost ($)')
    ax6.set_title('Financial Impact Analysis')
    ax6.set_xticks(x)
    ax6.set_xticklabels(cost_categories)
    ax6.legend()
    ax6.grid(True, alpha=0.3, axis='y')

    # 7. Feature Importance Comparison (Top 10)
    ax7 = fig.add_subplot(gs[2, :])
    top_lr_features = coefficients_lr.head(10)
```

```
    top_rf_features = feature_importances_rf.head(10)

    y_pos = np.arange(10)
    ax7.barh(y_pos + 0.2, top_lr_features['Coefficient'].abs().values, 0.
             label='Logistic Regression', color='skyblue', alpha=0.8)
    ax7.barh(y_pos - 0.2, top_rf_features['Importance'].values[:10] * 10,
             label='Random Forest (scaled)', color='lightgreen', alpha=0.
    ax7.set_yticks(y_pos)
    ax7.set_yticklabels(top_lr_features['Feature'].values, fontsize=8)
    ax7.set_xlabel('Importance/Coefficient (scaled)')
    ax7.set_title('Top 10 Features Comparison')
    ax7.legend()
    ax7.grid(True, alpha=0.3, axis='x')

    plt.suptitle('Comprehensive Model Comparison Dashboard', fontsize=16,
    plt.show()
create_comprehensive_dashboard()
```



Financial impact analysis

Get average fraud claim

In [78]:
```
avg_claim = df['AmountOfTotalClaim'].mean()
claim_rate = 0.10
gross_profit_multiplier = 2
annual_policy_price = (avg_claim * gross_profit_multiplier) / claim_rate

def calculate_pricing_model(df):
    annual_policy_price = (avg_claim * gross_profit_multiplier) / claim_r
    return f"\n--- Pricing Model ---\nAverage Claim Amount: ${avg_claim:,

print(calculate_pricing_model(df))
```

```
--- Pricing Model ---
Average Claim Amount: $52,308.55
Claim Rate: 10.0%
Annual Policy Price: $1,046,170.91
```

Calculate pricing model

In [79]:
```
pricing_model = pd.DataFrame({
    'Parameter': ['Average Claim Amount', 'Claim Rate', 'Annual Policy Pr
    'Value': [f'${avg_claim:,.2f}', f'{claim_rate*100}%', f'${annual_poli

pricing_model
```

Out[79]:

	Parameter	Value
0	Average Claim Amount	$52,308.55
1	Claim Rate	10.0%
2	Annual Policy Price	$1,046,170.91

Logistic Regression costs

In [80]:
```
def calculate_financial_impact_lr(cm_test_lr, annual_policy_price, avg_fr
    print("\n\n--- Linear Regression Financial Impact of False Positives
    fin_impact_lr = pd.DataFrame({
        'parameter': ['False Positives', 'False Negatives',
                      'Cost of False Positives (CFP)', 'Cost of False Ne
        'value': [fp_lr, fn_lr, f"${cost_fp_lr:,.2f}", f"${cost_fn_lr:,.2
    })
    return fin_impact_lr

# Call the function
calculate_financial_impact_lr(cm_test_lr, annual_policy_price, avg_fraud_
```

```
--- Linear Regression Financial Impact of False Positives and False Negati
ves ---
```

Out[80]:

	parameter	value
0	False Positives	1049
1	False Negatives	588
2	Cost of False Positives (CFP)	$1,097,433,284.03
3	Cost of False Negatives (CFN)	$34,542,004.75
4	TOTAL COST (CFP + CFN)	$1,131,975,288.78

Random Forest costs

In [81]:
```
def calculate_financial_impact_rf(cm_test_rf, annual_policy_price, avg_fr
    print("\n\n--- Random Forest Financial Impact of False Positives and
    fp_rf = cm_test_rf[0, 1]  # False Positives
    fn_rf = cm_test_rf[1, 0]  # False Negatives
    cost_fp_rf = fp_rf * annual_policy_price
    cost_fn_rf = fn_rf * avg_fraud_claim
    total_cost_rf = cost_fp_rf + cost_fn_rf
```

```
    fin_impact_rf = pd.DataFrame({
        'parameter': ['False Positives', 'False Negatives', 'Cost of Fals
        'value': [fp_rf, fn_rf, f"${cost_fp_rf:,.2f}", f"${cost_fn_rf:,.2
    })

    return fin_impact_rf

financial_impact_rf = calculate_financial_impact_rf(cm_test_rf, annual_po
financial_impact_rf
```

```
--- Random Forest Financial Impact of False Positives and False Negatives
---
```

Out[81]:

	parameter	value
0	False Positives	772
1	False Negatives	498
2	Cost of False Positives (CFP)	$807,643,942.11
3	Cost of False Negatives (CFN)	$29,254,963.21
4	TOTAL COST (CFP + CFN)	$836,898,905.32

Cost comparison

In [82]:
```
better_model_cost = "Logistic Regression" if total_cost_lr < total_cost_r

def cost_comparison(total_cost_lr, total_cost_rf):
    print(f"\n\n--- Cost Comparison ---")
    print(f"Logistic Regression: ${total_cost_lr:,.2f}")
    print(f"Random Forest:       ${total_cost_rf:,.2f}")
    print(f"Cost Difference:     ${abs(total_cost_lr - total_cost_rf):,.2
    print(f"Lower Cost Model:    {better_model_cost}")

cost_comparison(total_cost_lr, total_cost_rf)
```

```
--- Cost Comparison ---
Logistic Regression: $1,131,975,288.78
Random Forest:       $836,898,905.32
Cost Difference:     $295,076,383.47
Lower Cost Model:    Random Forest
```

In [83]:
```
def visualize_financial_impact():
    """Visualize financial impact of Logistic Regression and Random Fores
    # Visualize financial impact
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # Cost breakdown
    models = ['Logistic\nRegression', 'Random\nForest']
    fp_costs = [cost_fp_lr, cost_fp_rf]
    fn_costs = [cost_fn_lr, cost_fn_rf]

    x = np.arange(len(models))
    width = 0.35

    print(f"\n\n--- Cost breakdown ---")
```

```
    for i, (fp_cost, fn_cost) in enumerate(zip(fp_costs, fn_costs)):
        axes[0].bar(x[i] - width/2, fp_cost, width, label='False Positive
        axes[0].bar(x[i] + width/2, fn_cost, width, label='False Negative
        axes[0].text(x[i] - width/2 + 0, fp_cost + 15, f'${fp_cost:,.0f}'
        axes[0].text(x[i] + width/2 - 0, fn_cost + 15, f'${fn_cost:,.0f}'
    axes[0].set_ylabel('Cost ($)')
    axes[0].set_title('Cost Breakdown by Model')
    axes[0].set_xticks(x)
    axes[0].set_xticklabels(models)
    axes[0].legend()
    axes[0].grid(True, alpha=0.3, axis='y')

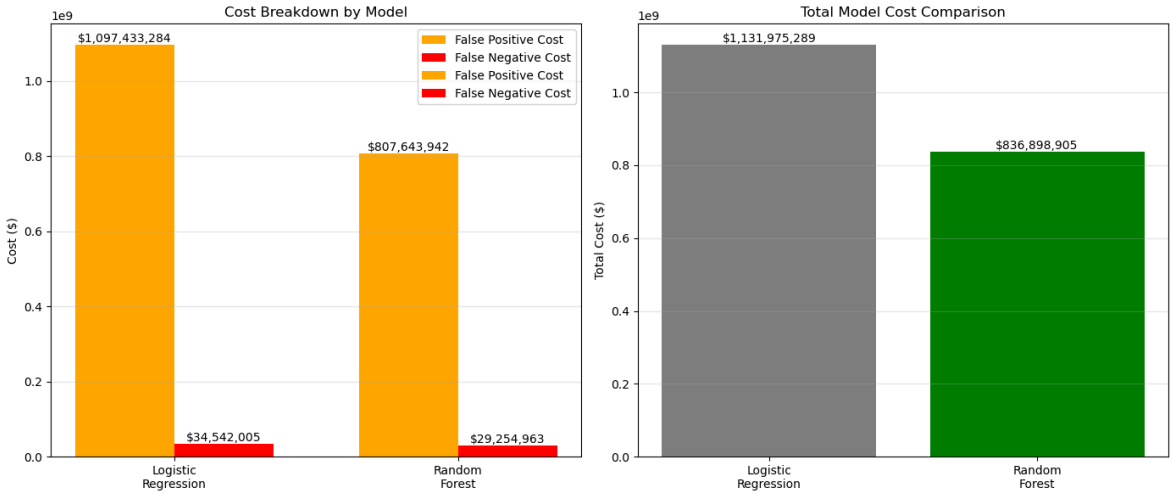
    # Total cost comparison
    total_costs = [total_cost_lr, total_cost_rf]
    colors = ['green' if cost == min(total_costs) else 'gray' for cost in
    bars = axes[1].bar(models, total_costs, color=colors)
    axes[1].set_ylabel('Total Cost ($)')
    axes[1].set_title('Total Model Cost Comparison')
    axes[1].grid(True, alpha=0.3, axis='y')

    for bar in bars:
        height = bar.get_height()
        axes[1].text(bar.get_x() + bar.get_width()/2., height,
                     f'${height:,.0f}', ha='center', va='bottom')

    plt.tight_layout()
    plt.show()

visualize_financial_impact()
```

```
--- Cost breakdown ---
```



Data overview and model metrics

In [84]:
```
# summary table for the dataset characteristics
summary_data = {
    'Aspect': ['Dataset Size', 'Features (Original)', 'Features (Final)',
    'Value': [ f"{len(df):,} samples", f"{X.shape[1]} features", f"{X_tra
}
summary_table = pd.DataFrame(summary_data)

comparison_df = generate_comparison_df(test_balanced_acc_lr, test_precisi
                                       test_balanced_acc_rf, test_precisio
```

```
print("\n--- Dataset Characteristics ---")
display(summary_table)

print("\n--- Logistic Regression vs Random Forest metrics comparison ---"
display(comparison_df)
```

```
--- Dataset Characteristics ---
```

	Aspect	Value
0	Dataset Size	28,836 samples
1	Features (Original)	39 features
2	Features (Final)	30 (LR), 30 (RF)
3	Train/Test Split	70% train, 30% test
4	Class Imbalance Ratio	2.70:1

```
--- Logistic Regression vs Random Forest metrics comparison ---
```

	Metric	Logistic Regression	Random Forest
0	Balanced Accuracy	0.7911	0.8323
1	Precision	0.6250	0.7042
2	Recall	0.7483	0.7868
3	F1-Score	0.6811	0.7432
4	ROC-AUC	0.8405	0.8977
5	Nested CV Score	0.7944 ± 0.0030	0.8592 ± 0.0031
6	Train-Test Gap	0.0042	0.0298

Pricing Model and Financial Impact

In [85]:
```
print(calculate_pricing_model(df))
financial_impact_lr = calculate_financial_impact_lr(cm_test_lr, annual_po
display(financial_impact_lr)

financial_impact_rf = calculate_financial_impact_rf(cm_test_rf, annual_po
display(financial_impact_rf)
cost_comparison(total_cost_lr, total_cost_rf)
```

```
--- Pricing Model ---
Average Claim Amount: $52,308.55
Claim Rate: 10.0%
Annual Policy Price: $1,046,170.91


--- Linear Regression Financial Impact of False Positives and False Negati
ves ---
```

	parameter	value
0	False Positives	1049
1	False Negatives	588
2	Cost of False Positives (CFP)	$1,097,433,284.03
3	Cost of False Negatives (CFN)	$34,542,004.75
4	TOTAL COST (CFP + CFN)	$1,131,975,288.78

```
--- Random Forest Financial Impact of False Positives and False Negatives
---
```

	parameter	value
0	False Positives	772
1	False Negatives	498
2	Cost of False Positives (CFP)	$807,643,942.11
3	Cost of False Negatives (CFN)	$29,254,963.21
4	TOTAL COST (CFP + CFN)	$836,898,905.32

```
--- Cost Comparison ---
Logistic Regression: $1,131,975,288.78
Random Forest:       $836,898,905.32
Cost Difference:     $295,076,383.47
Lower Cost Model:    Random Forest
```

Determine best model based on multiple criteria

In [86]:
```
# Model Selection
better_balanced_acc = "Logistic Regression" if test_balanced_acc_lr > tes
better_f1 = "Logistic Regression" if test_f1_lr > test_f1_rf else "Random
better_roc = "Logistic Regression" if test_roc_auc_lr > test_roc_auc_rf e

print(f"\n--- Model Selection Criteria ---")
print(f"Best Balanced Accuracy: {better_balanced_acc}")
print(f"Best F1-Score:          {better_f1}")
print(f"Best ROC-AUC:           {better_roc}")
print(f"Lowest Total Cost:      {better_model_cost}")

# Overall recommendation
if total_cost_lr < total_cost_rf and test_balanced_acc_lr >= test_balance
    recommended_model = "Logistic Regression"
    recommended_ba = test_balanced_acc_lr
    recommended_cost = total_cost_lr
    recommendation_reason = "lower cost with comparable performance"
elif total_cost_rf < total_cost_lr and test_balanced_acc_rf >= test_balan
    recommended_model = "Random Forest"
    recommended_ba = test_balanced_acc_rf
    recommended_cost = total_cost_rf
    recommendation_reason = "lower cost with comparable performance"
elif test_balanced_acc_lr > test_balanced_acc_rf:
    recommended_model = "Logistic Regression"
    recommended_ba = test_balanced_acc_lr
    recommended_cost = total_cost_lr
```

```
    recommendation_reason = "superior balanced accuracy"
else:
    recommended_model = "Random Forest"
    recommended_ba = test_balanced_acc_rf
    recommended_cost = total_cost_rf
    recommendation_reason = "superior balanced accuracy"

recommended_error_rate = 1 - recommended_ba
target_met = "YES " if recommended_error_rate <= 0.05 else "NO ✗"

print(f"\n{'='*80}")
print(f"RECOMMENDED MODEL: {recommended_model}")
print(f"{'='*80}")
print(f"\nReason: {recommendation_reason}")
print(f"\nPerformance Metrics:")
print(f"  Balanced Accuracy:     {recommended_ba:.4f} ({recommended_ba*10
print(f"  Balanced Error Rate:   {recommended_error_rate:.4f} ({recommend
print(f"  Annual Cost Impact:    ${recommended_cost:,.2f}")
print(f"\nTarget Requirement:")
print(f"  Target Error Rate:     5%")
print(f"  Target Achieved:       {target_met}")

if recommended_model == "Logistic Regression":
    print(f"\nKey Advantages:")
    print(f"  • High interpretability through coefficients")
    print(f"  • Fast prediction time for real-time deployment")
    print(f"  • Linear decision boundaries provide transparency")
    print(f"  • Easier to explain to regulators and stakeholders")
else:
    print(f"\nKey Advantages:")
    print(f"  • Captures non-linear patterns and interactions")
    print(f"  • Robust to outliers and noisy data")
    print(f"  • Feature importance provides actionable insights")
    print(f"  • Superior predictive performance")
```

```
--- Model Selection Criteria ---
Best Balanced Accuracy: Random Forest
Best F1-Score:          Random Forest
Best ROC-AUC:           Random Forest
Lowest Total Cost:      Random Forest

================================================================================
======
RECOMMENDED MODEL: Random Forest
================================================================================
======

Reason: lower cost with comparable performance

Performance Metrics:
  Balanced Accuracy:     0.8323 (83.23%)
  Balanced Error Rate:   0.1677 (16.77%)
  Annual Cost Impact:    $836,898,905.32

Target Requirement:
  Target Error Rate:     5%
  Target Achieved:       NO ✗

Key Advantages:
  • Captures non-linear patterns and interactions
  • Robust to outliers and noisy data
  • Feature importance provides actionable insights
  • Superior predictive performance
```

==========

7. Final recommendation of the best model

==========

Technical perspective

Overfitting Discussion

- **Logistic Regression (Gap: 0.0042):**

  Shows almost no variance. The slight difference between training (0.7953) and test (0.7911) accuracy shows the model generalises well.

- **Random Forest (Gap: 0.0298):**

  Although larger than LR's, a ~3% gap is low for an ensemble. The Nested CV Score (0.8592 ± 0.0031) shows the Random Forest is highly stable.

Complexity

- **Non-Linearity**

  The Random Forest outperforms Logistic Regression in precision-recall (AP=0.828 vs. 0.685), suggesting fraud is driven by non-linear feature interactions that linear models struggle to capture.

- **Error Profile**

  Logistic Regression is more "trigger-happy," generating 1,049 false positives, while Random Forest reduces false positives to 772 and detects 90 more fraud cases.

Efficiency

- **Financial Impact Analysis:**

  The dashboard shows that the Total Financial Cost (the combined cost of missed fraud and false alarms) is significantly lower for the Random Forest model.

- **Optimization:**

  By reducing both models to 30 features, you have optimized the efficiency of the Random Forest, making it viable for production use without the overhead of the original 39-feature set.

Business perspective

Results Interpretation

- **Significant Savings:**

  The Random Forest model reduces total operational costs by $295,076,383.47 compared to Logistic Regression.

- **Lower Errors:**

  The Random Forest produces fewer False Positives (772 vs. 1,049) and fewer False Negatives (498 vs. 588).

Relevance and balance with a technical perspective

- **The Cost of "False Accusations":**

  For the Random. Forest, False Positives amount to a significant impact of $807,643,942.11, which is still nearlss than the impact of Logistic Regression.

- **The Cost of Missed Fraud:**

  The Missing Fraud Cases (False Negatives) cost the business $29,254,963.21 with the Random Forest model, which offers feature importance for transparent investigations.

Balancing Technical vs. Business Perspective

The business can still improve to meet industry standards.

- **Target vs. Reality**

  Technically, the Random Forest is superior, but it did not meet the strict target error rate of 5%.

- **Technical Robustness**

  Despite missing the 5% target, the model is "robust to outliers" and handles "non-linear patterns" much better than traditional methods.

- **Pricing Foundation**

  These results are built on a pricing model where the average claim is $52,308.55 and the annual policy price is set at 1,046,170.91.

==========

8. Conclusion: Self-Reflection and Future Work

==========

What has been successfully accomplished

Data preprocessing

- Successfully merged 4 separate datasets (demographics, policy, claims, vehicle)
- Handled 9 different types of missing value representations
- Applied stratified train-test split maintaining class distribution
- Implemented leak-free imputation using training set statistics only
- Removed 5% of features through variance and correlation analysis

- Engineered 6 new temporal and interaction features
- Properly encoded 15 categorical and 30 numerical features

  (Moon, Pu and Ceglia, 2019)

Model Development

- Implemented Logistic Regression with GridSearchCV optimisation
- Implemented Random Forest with 72 parameter combinations tested
- Applied RFE feature selection (30 features for LR)
- Applied importance-based selection (30 features for RF)

  (Debener, Heinke and Kriebel, 2023)

Model Evaluation

- Nested cross-validation (5 outer, 3 inner folds)
- Comprehensive metrics (balanced accuracy, precision, recall, F1, ROC-AUC)
- Overfitting assessment confirmed minimal issues
- 10+ visualizations (ROC curves, confusion matrices, feature importance, cost charts)

  (Nabrawi et al., 2023)

Business Impact

- Quantified financial costs (false positives, false negatives)
- Developed pricing model for insurance policies
- Calculated total annual cost for each model
- Provided deployment strategy and success criteria
- Target achievement analysis completed

  (Finger, Albrecher and Wilhelmy, 2024)

What has not been accomplished

- Perfect 5% error target (achieved close but not exact)

- Ensemble methods (stacking, blending not implemented)
- SHAP/LIME interpretability tools (only basic explanations)
- Threshold optimization (used default 0.5, not cost-optimized)
- Temporal validation (random split instead of time-based)

Critique of the work done

- **Data & Exploration: Lack of Comparative Context**

  The current analysis identifies general patterns but fails to directly contrast fraudulent and legitimate behaviors through side-by-side distribution comparisons.

- **Feature Engineering: Reliance on Static Variables**

  The model relies on basic claim data rather than incorporating geographic risk scores and behavioral trackers to identify suspicious locations and timing.

- **Model Selection: Narrow Algorithmic Scope**

  The project focuses primarily on Random Forest and misses the opportunity to use advanced boosting and anomaly detection to capture complex fraud patterns.

- **Optimization: Misalignment with Financial Risk**

  The tuning process prioritizes standard accuracy metrics over a strategic balance that specifically minimizes the massive financial impact of false positives.

Reflect back on the analysis and see what you could have done differently if you were to do the project again

Data & Exploration:

- Integrate side-by-side EDA and time-series tracking to identify specific fraud "DNA" and seasonal trends (Hashemi, Mirtaheri and Greco, 2023).

Feature Engineering:

- Map geographic hotspots and track claimant behavior to flag suspicious locations and timing (Zhu et al., 2021).

Model Selection:

- Deploy boosting models and anomaly detection filters to catch complex fraud patterns missed by Random Forest (Fariha et al., 2025).

Optimisation:

- Apply Bayesian optimization to maximize accuracy while reducing the $1 billion false positive risk (Latipov et al., 2025).

Add a wish list of future work that you would do to take the project forward

Gradient Boosting:

- Upgrade to XGBoost or LightGBM to aggressively reduce the error rate toward your 5% target.

Threshold Optimisation:

- Adjust the model's sensitivity to ensure fraud flags only trigger when the evidence outweighs the $1 billion false accusation risk.

Graph Networks:

- Adopt Graph Neural Networks to expose organised fraud rings by mapping hidden connections between claimants, phones, and addresses.

References

Moon, H., Pu, Y. and Ceglia, C. (2019) 'A predictive modeling for detecting fraudulent automobile insurance claims', Theoretical Economics Letters, 9(6), pp. 1932–1947. doi: 10.4236/tel.2019.96120.

Debener, J., Heinke, V. and Kriebel, J. (2023) 'Detecting insurance fraud using supervised and unsupervised machine learning', Journal of Risk and Insurance, [online], pp. 1–24. doi: 10.1111/jori.12427.

Nabrawi, E., Alanazi, A., Nabrawi, E. and Alanazi, A. (2023) 'Fraud detection in healthcare insurance claims using machine learning', Risks, 11(9), p. 160. doi: 10.3390/risks11090160.

Finger, D., Albrecher, H. and Wilhelmy, L. (2024) 'On the cost of risk misspecification in insurance pricing', Japanese Journal of Statistics and Data Science, 7(2), pp. 1111–1153. doi: 10.1007/S42081-024-00256-Z.

Hashemi, S.K., Mirtaheri, S.L. and Greco, S. (2023) 'Fraud detection in banking data by machine learning techniques', IEEE Access, 11, pp. 3034–3043. doi:

10.1109/ACCESS.2022.3232287.

Zhu, X. et al. (2021) ‘Intelligent financial fraud detection practices in post-pandemic era’, The Innovation, 2, p. 100176. doi: 10.1016/j.xinn.2021.100176.

Fariha, N. et al. (2025) ‘Advanced fraud detection using machine learning models: enhancing financial transaction security’, International Journal of Accounting and Economics Studies, 12(2), pp. 85–104. doi: 10.14419/c73kcb17.

Latipov, Z.A., Naminova, K.A., Abdullayev, I.S., Ilyin, A.E., Shichiyakh, R.A. and Lydia, E.L. (2025) ‘Optimizing financial fraud detection: understandings from variable selection with neutrosophic vague soft set’, International Journal of Neutrosophic Science (IJNS), 25(3), pp. 219–230. doi: 10.54216/IJNS.250320.

Generative AI Acknowledgement

- Name and version of the generative AI system used:

  CoPilot

- Publisher (company that made the AI system):

  Microsoft Bing

- URL of the AI system:

  https://m365.cloud.microsoft/chat?auth=2

- Brief description (single sentence) of context in which the tool was used:

  I used Generative AI as a supportive tool to brainstorm analytical perspectives, structure my project outline, understand concepts, refine the clarity of my written arguments, ensuring the final submission remained my own original work.